Water

Chi Y. Cheng, Maciej Bartkowiak, Sanghamitra Mukhopadhyay

MDANSE Workshop 2026

04/11/2026



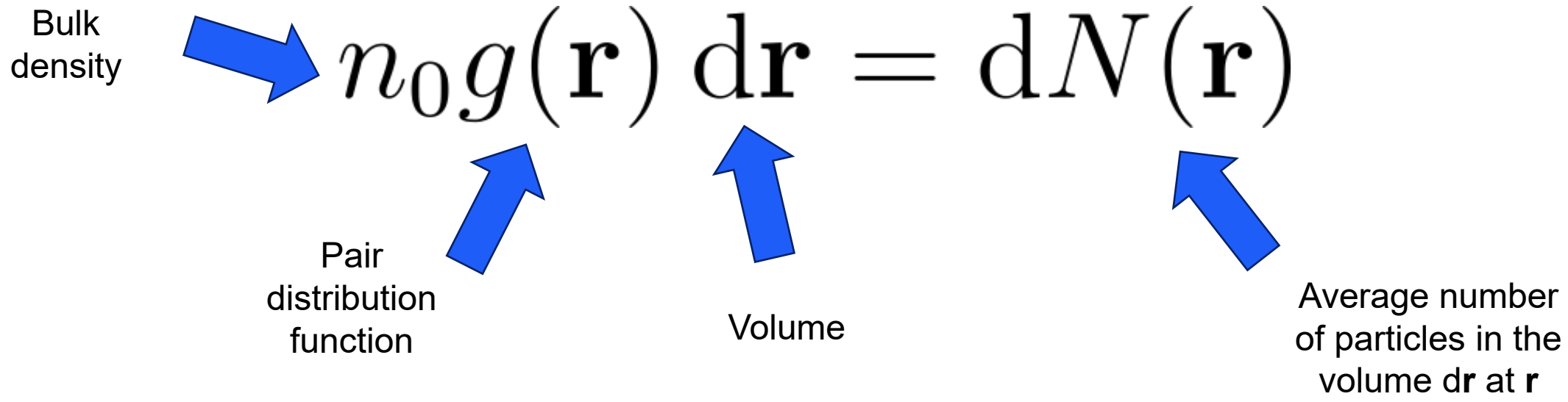
Water

- In this workshop exercise, we will run simulation of water using OpenMM and then run van Hove calculations with MDANSE.
- But what is the van Hove function?



Pair Distribution Function

- Let's start with the pair distribution function which provides us with some information about how atoms are distributed in the system.
- It tells us what the average number of particles will be at a distance **r** in volume d**r** from an atom.



Pair Distribution Function

- Let's start with the pair distribution function which provides us with some information about how atoms are distributed in the system.
- In this case the pair distribution function tells us the average number of particles in the shell volume $4\pi r^2\,dr$ from a distance r of an atom.

Bulk density

$$n_0 g(r)\, 4\pi r^2\, \mathrm{d}r = \mathrm{d}N(r)$$

Pair distribution function

Shell volume

Average number of particles in the shell between r and r + dr

Pair Distribution Function

- The PDF can be written as a sum of delta functions.

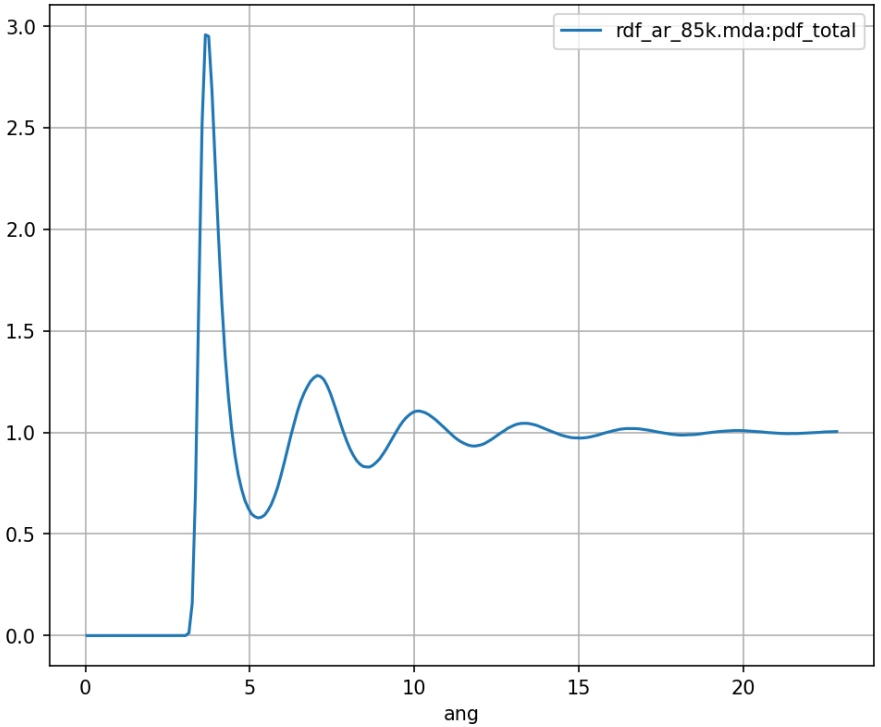
$$n_0 g(r)\, 4\pi r^2\, \mathrm{d}r = \mathrm{d}N(r)$$

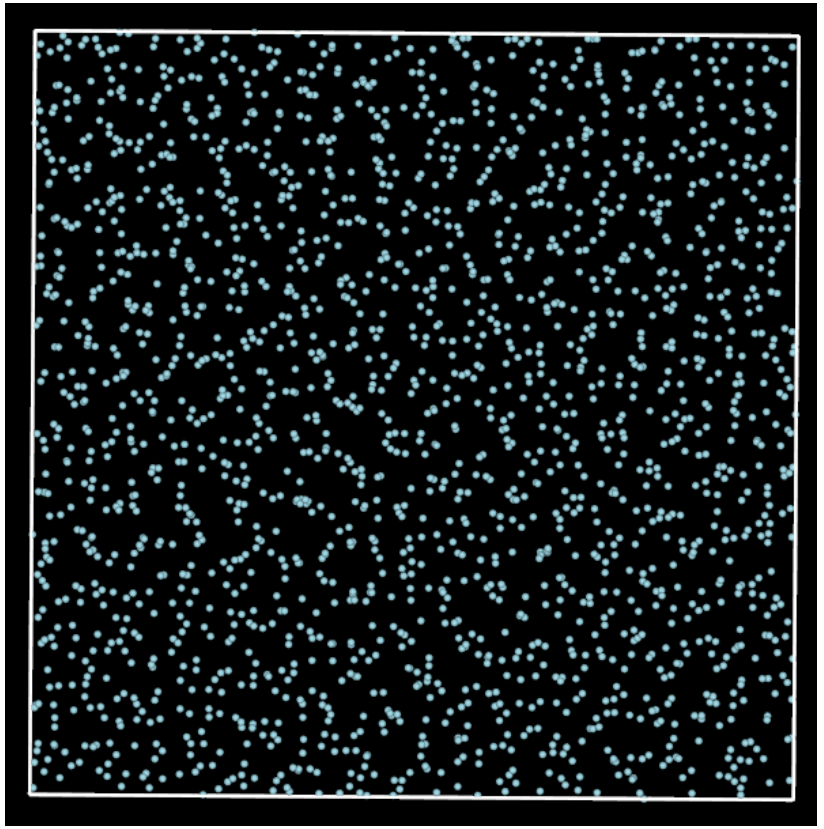
$$n_0 g(r) = \frac{1}{4\pi r^2}\frac{1}{N}\sum_{k\neq j}\left\langle \delta[r - |\mathbf{r}_k(t_0) - \mathbf{r}_j(t_0)|]\right\rangle_{t_0}$$

Pair Distribution Function

- Here is the PDF of a liquid argon trajectory generated from MDANSE.

This Ar trajectory was generated using lammps at 85 K with the Lennard-Jones potential.

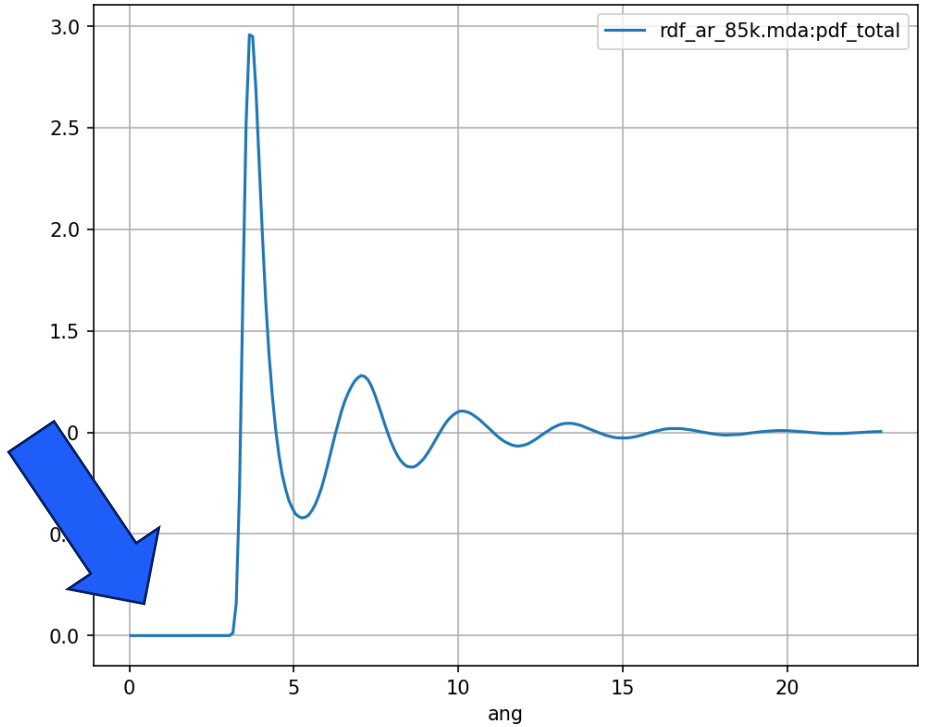


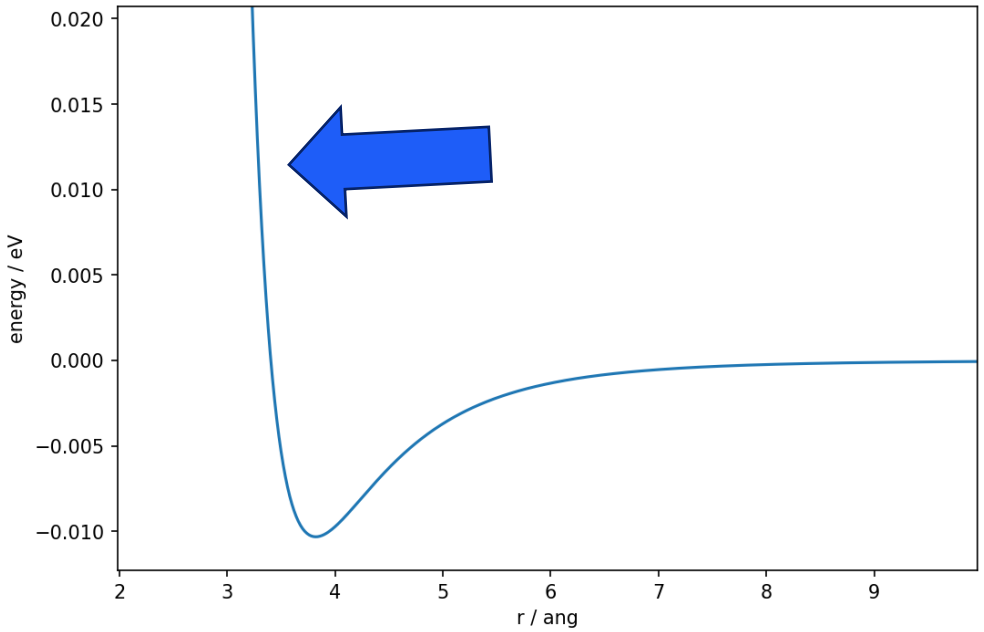


Pair Distribution Function

- Here is the PDF of a liquid argon trajectory generated from MDANSE.

We can see the repulsive $1/r^{12}$ potential causing a reduction of particles close to each atom.

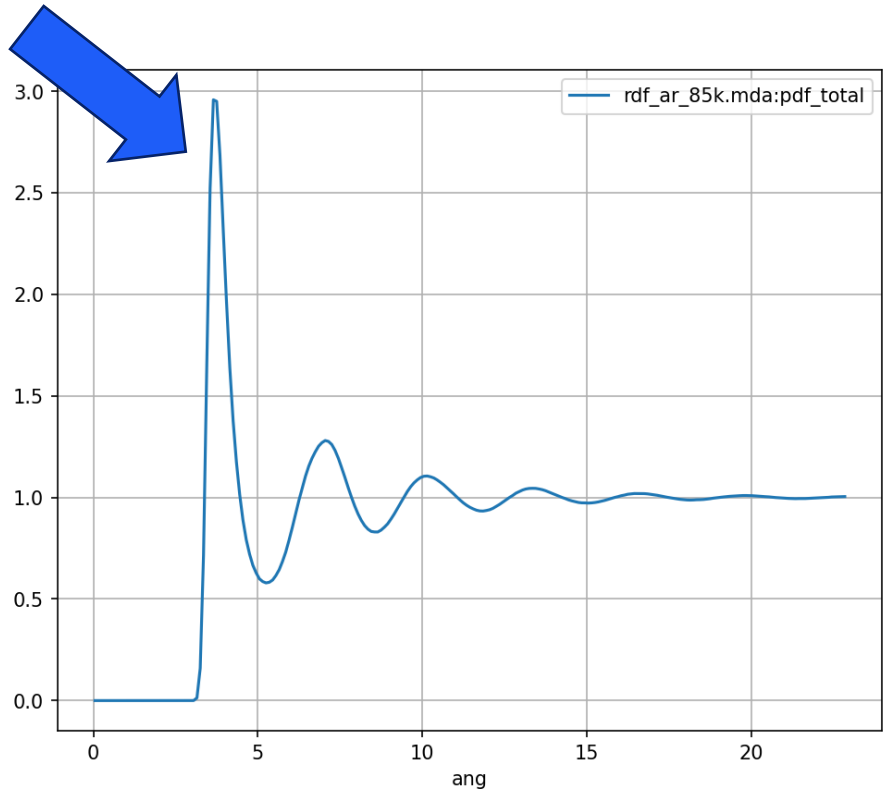


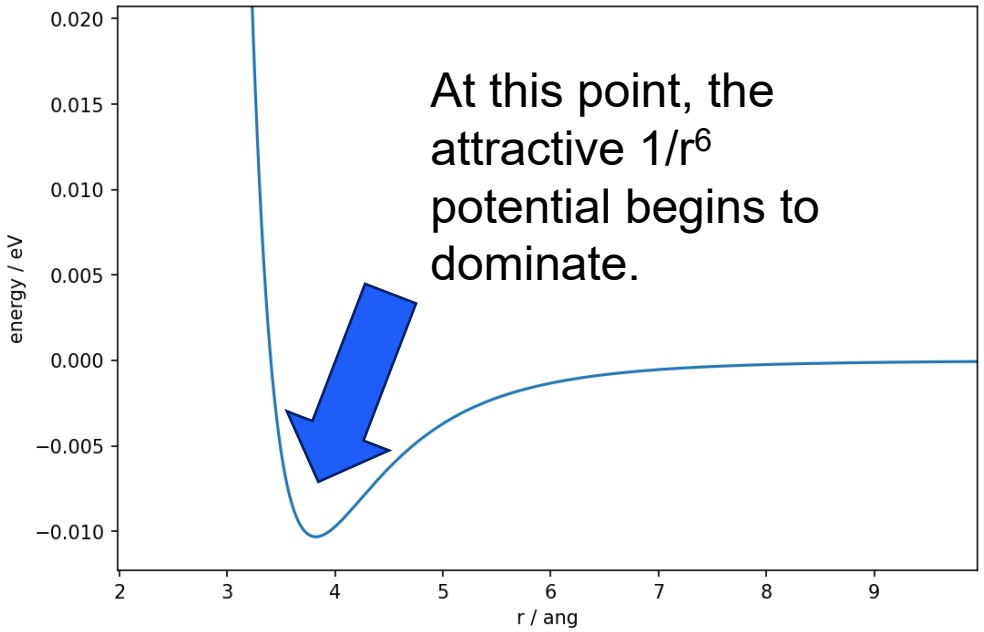


Pair Distribution Function

- Here is the PDF of a liquid argon trajectory generated from MDANSE.

The first peak in the PDF occurs around 3.7 Å this is approximately the minimum of the Lennard-Jones potential 3.8 Å.

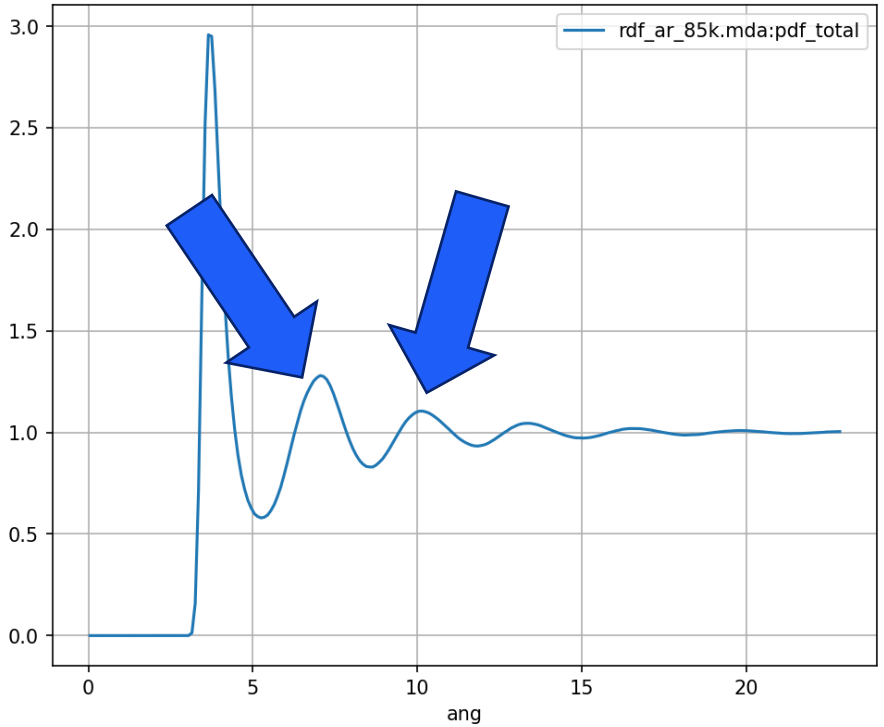


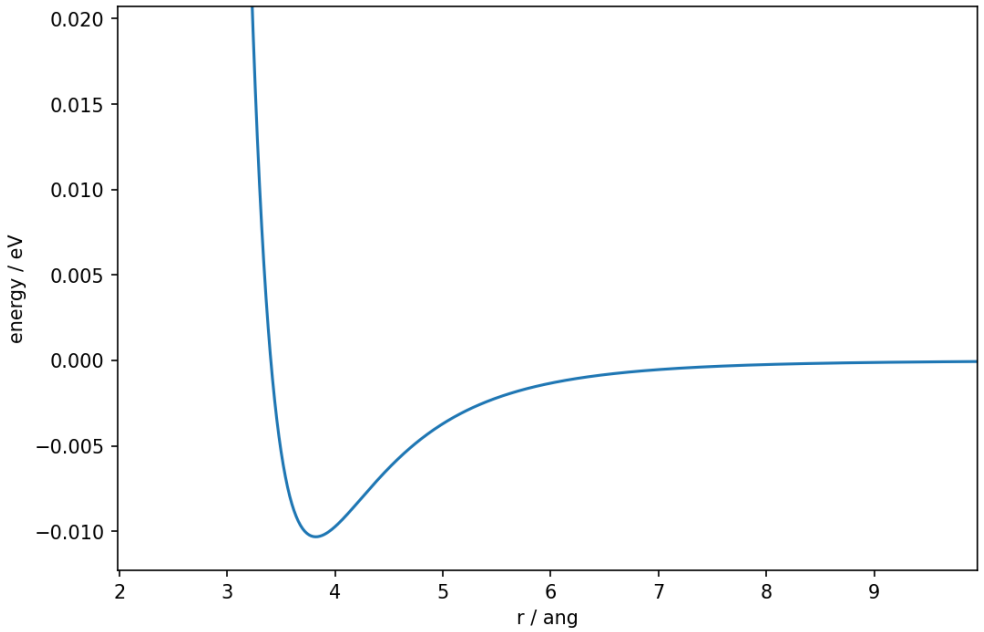


Pair Distribution Function

- Here is the PDF of a liquid argon trajectory generated from MDANSE.

The other peaks are the 2nd, 3rd and other shell of atoms around each atoms.

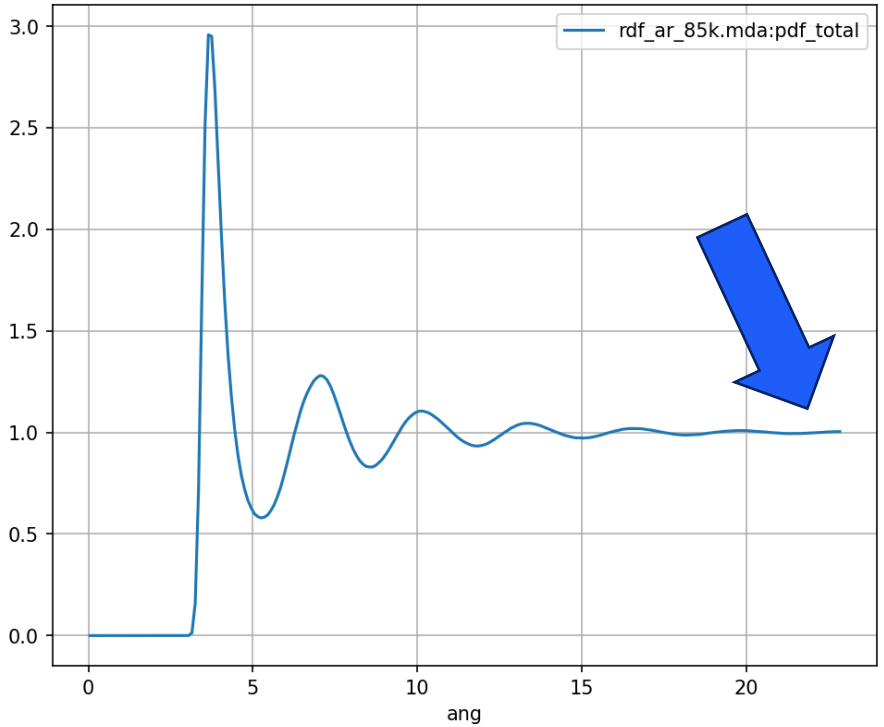


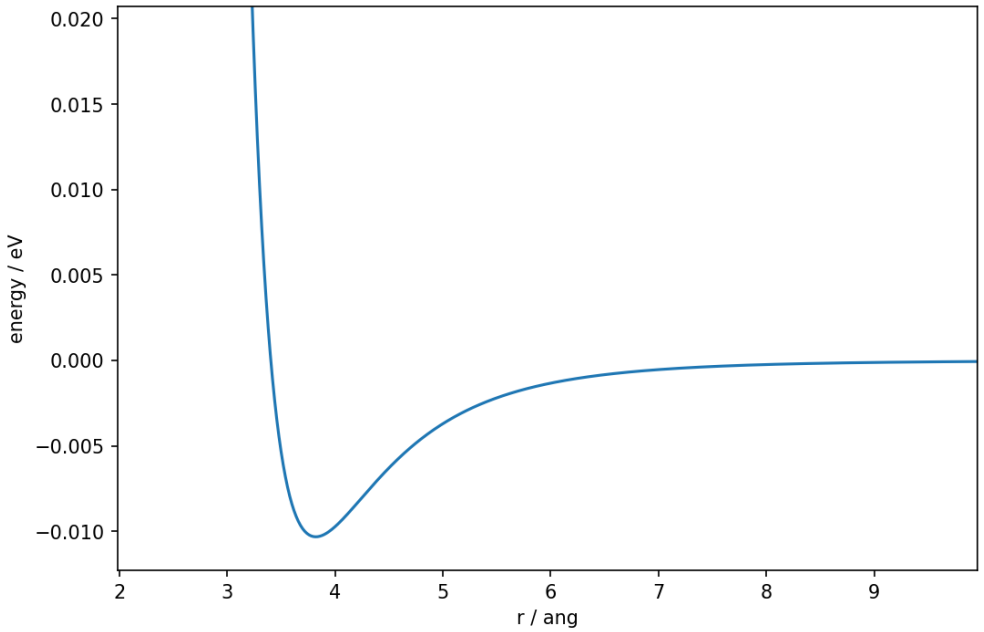


Pair Distribution Function

- Here is the PDF of a liquid argon trajectory generated from MDANSE.

For gas and liquid systems for large distance the PDF is normalized so that it goes to 1.

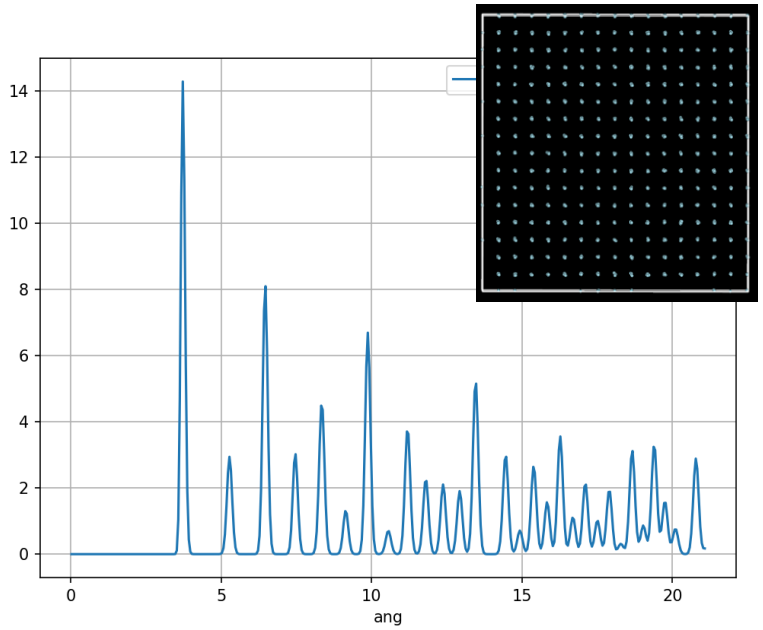




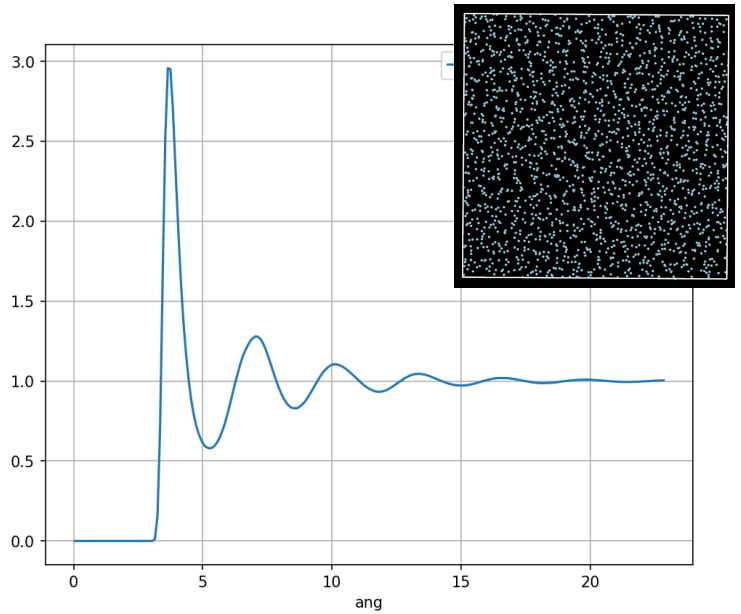
Pair Distribution Function

- The PDF can be quite different when the system exist in a different phase.

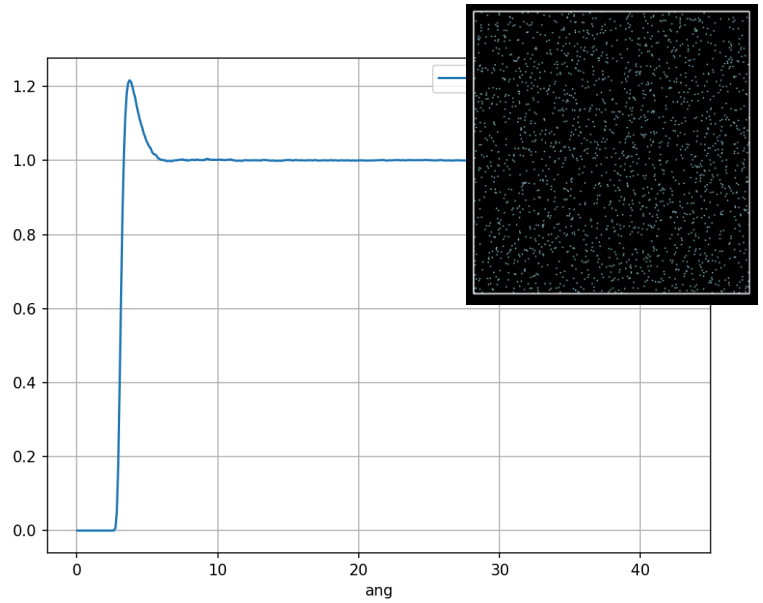
Solid (ordered)



Liquid



Gas



Van Hove Function

$$n_0 g(\mathbf{r}) = \frac{1}{N} \sum_{j \neq k} \langle \delta[\mathbf{r} - \mathbf{r}_k(t_0) + \mathbf{r}_j(t_0)] \rangle_{t_0}$$

- Similarly to the PDF the van Hove function is also a sum of delta function.

$$G(\mathbf{r}, t) = \frac{1}{N} \sum_{jk} \langle \delta[\mathbf{r} - \mathbf{r}_k(t_0 + t) + \mathbf{r}_j(t_0)] \rangle_{t_0}$$

- The PDF gives use some insights into the structure of our system while the van Hove function gives us insights into the structure and dynamics of the system.

Van Hove Function

- The van Hove function can be split in self and distinct terms

$$G_{\mathrm{s}}(\mathbf{r},t) = \frac{1}{N}\sum_{j}\left\langle \delta\left[\mathbf{r} - \mathbf{r}_j(t_0+t) + \mathbf{r}_j(t_0)\right]\right\rangle_{t_0}$$

$$G_{\mathrm{d}}(\mathbf{r},t) = \frac{1}{N}\sum_{j\neq k}\left\langle \delta\left[\mathbf{r} - \mathbf{r}_k(t_0+t) + \mathbf{r}_j(t_0)\right]\right\rangle_{t_0}$$

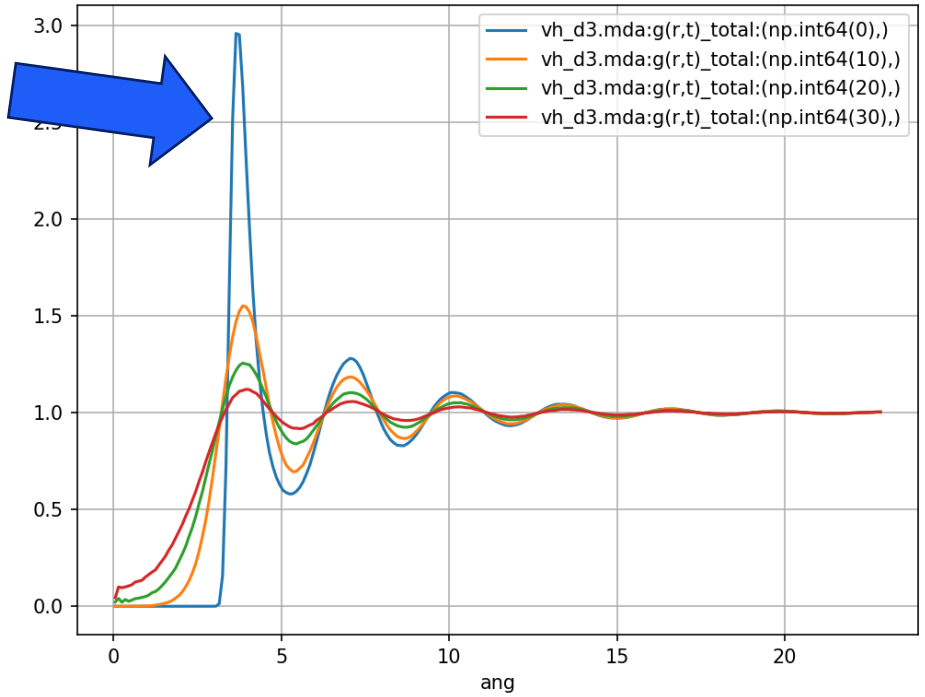
Van Hove Function (distinct-part)

- The distinct-part of the van Hove function can be written so that it is only a function of the distance r.

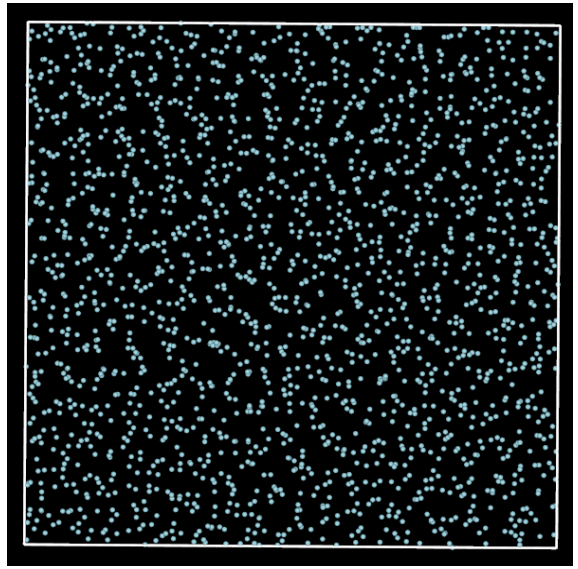
$$G_{\mathrm{d}}(r,t) = \frac{1}{4\pi r^2}\frac{1}{N}\sum_{j\neq k}\langle\delta[r-|\mathbf{r}_k(t_0+t)-\mathbf{r}_j(t_0)|]\rangle_{t_0}$$

At t=0 the distinct-part of the van Hove function is the PDF.

$$G_{\mathrm{d}}(\mathbf{r},0) = n_0 g(\mathbf{r})$$

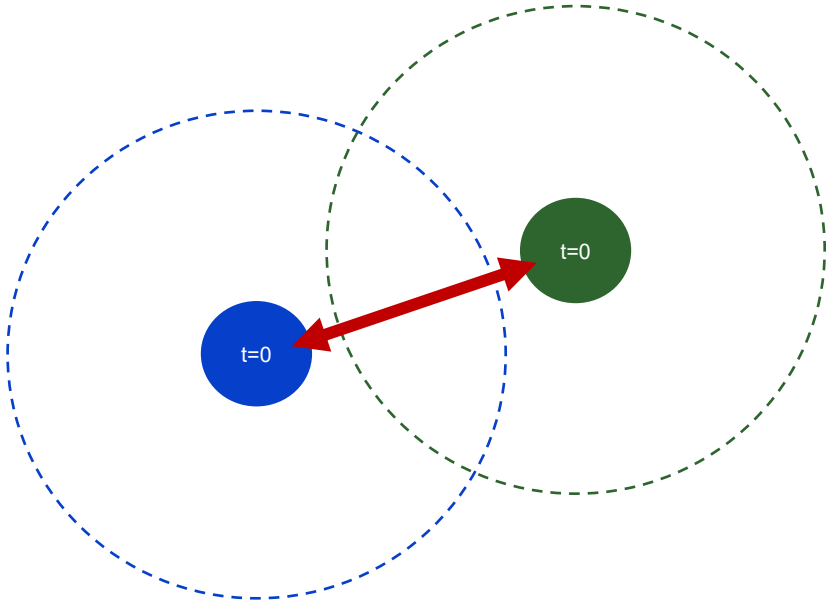


This is the liquid Ar MD trajectory at 85K.

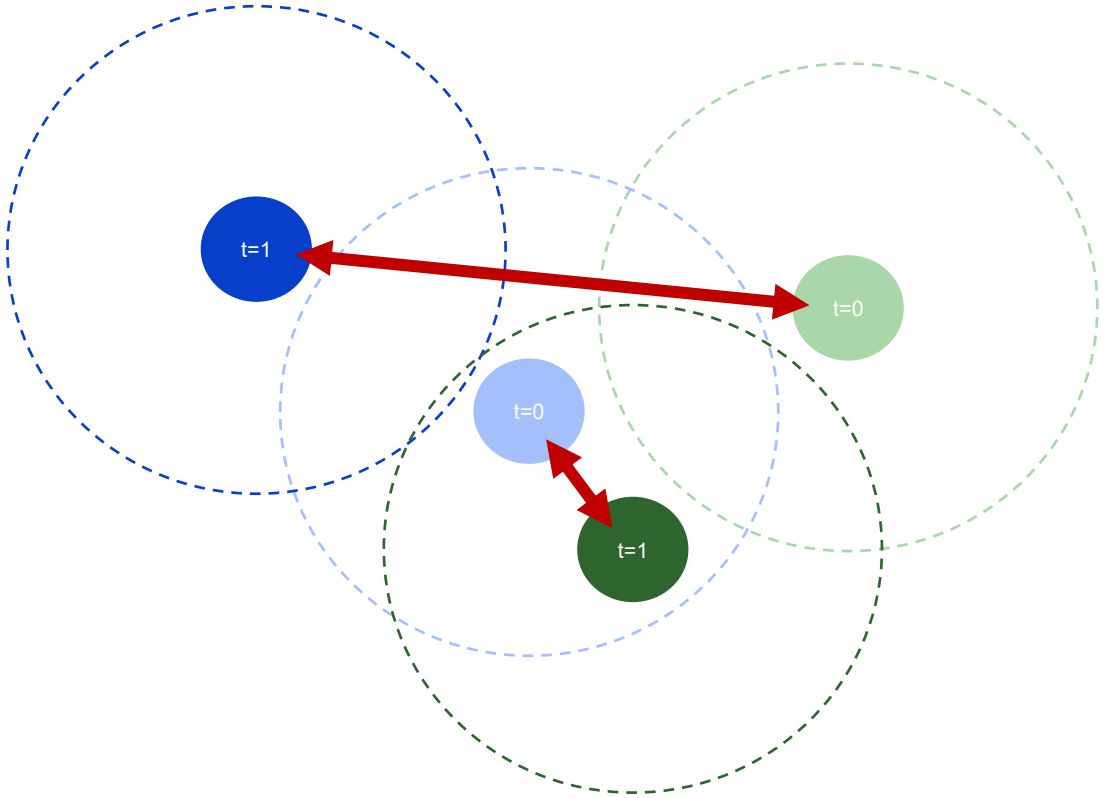


Van Hove Function (distinct-part)

$$G_{\mathrm{d}}(\mathbf{r}, t = 0)$$



$$G_{\mathrm{d}}(\mathbf{r}, t = 1)$$

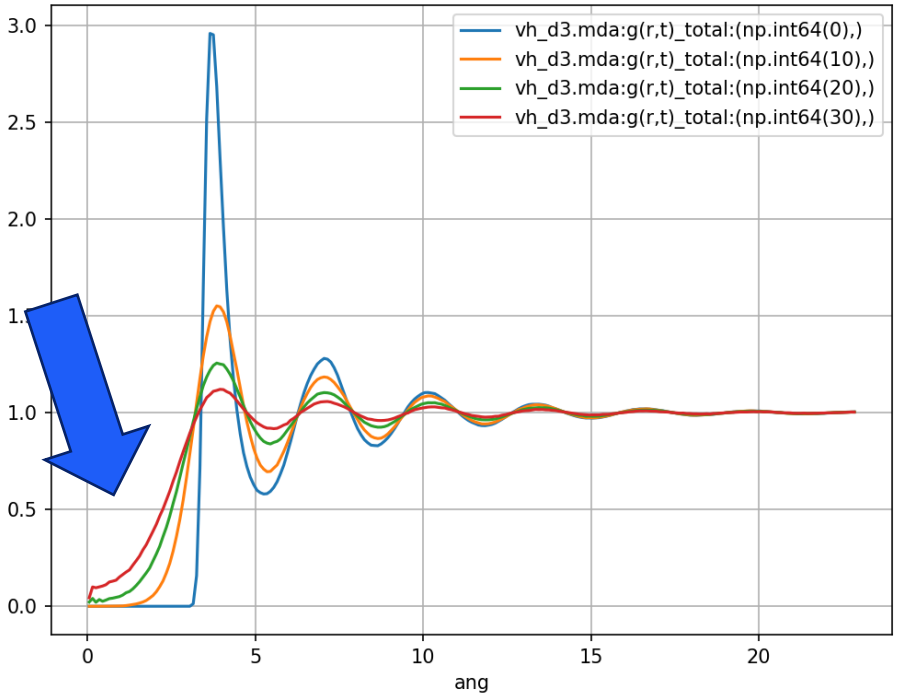


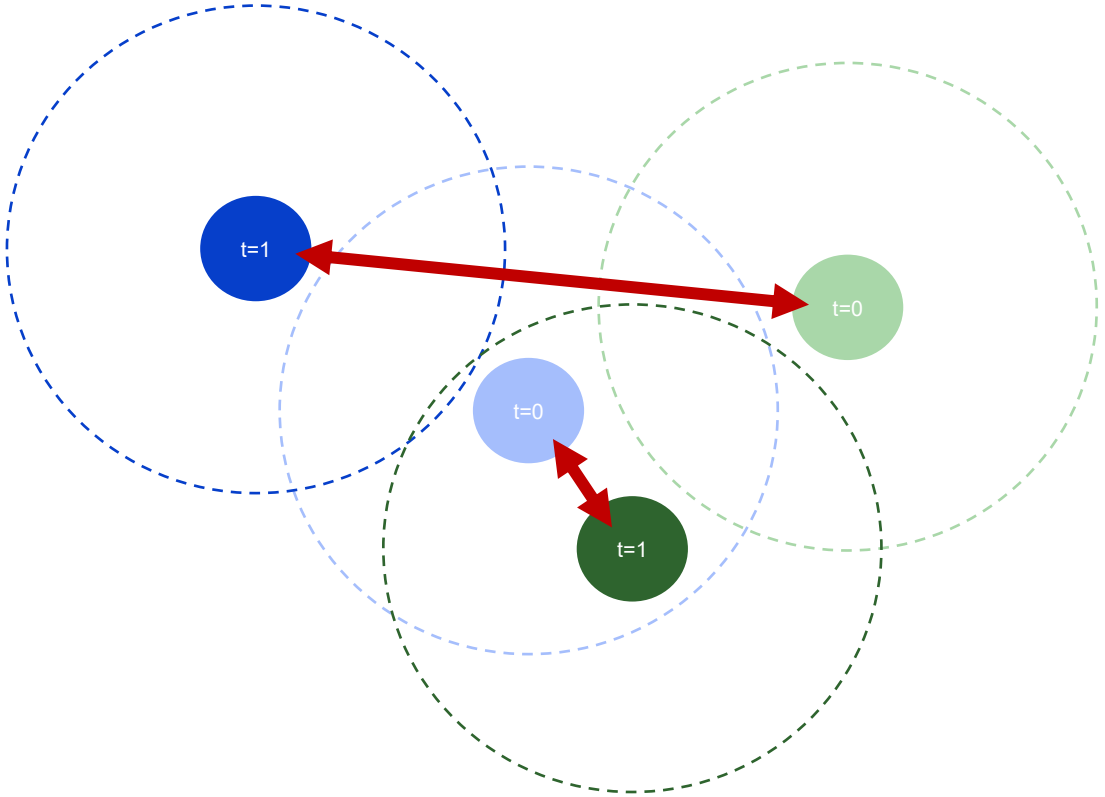
Van Hove Function (distinct-part)

- The distinct-part of the van Hove function can be written so that it is only a function of the distance r.

$$G_{\mathrm{d}}(r,t) = \frac{1}{4\pi r^2}\frac{1}{N}\sum_{j\neq k}\langle\delta[r-|\mathbf{r}_k(t_0+t)-\mathbf{r}_j(t_0)|]\rangle_{t_0}$$

As time progress the particle (j) moves away from its initial position and the hole that was around that particle begin to be occupied by other atoms.





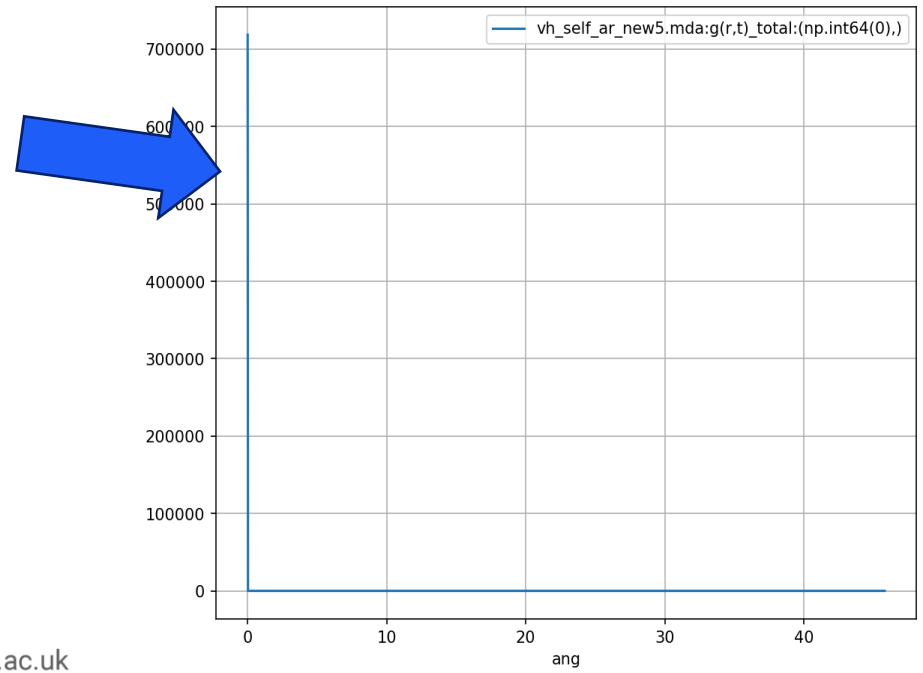
Van Hove Function (self-part)

- The self-part of the van Hove function can be written so that it is only a function of the distance r.

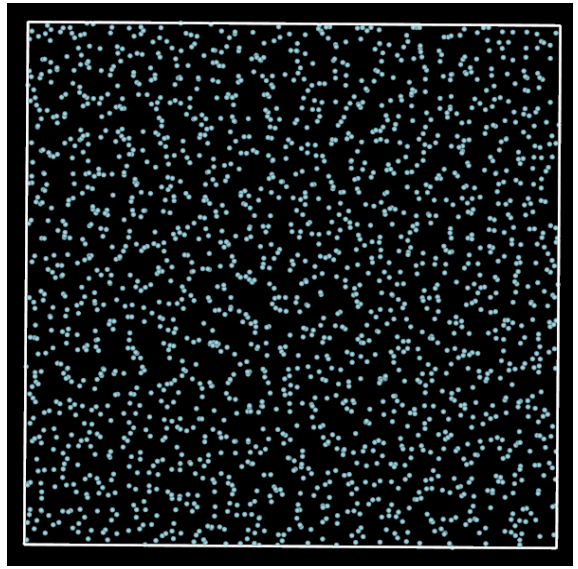
$$G_s(r,t) = \frac{1}{4\pi r^2}\frac{1}{N}\sum_j \langle \delta[r - |\mathbf{r}_j(t_0+t) - \mathbf{r}_j(t_0)|]\rangle_{t_0}$$

At t=0 the self-part of the van Hove function is a delta function.

$$G_s(\mathbf{r},0) = \delta(\mathbf{r})$$

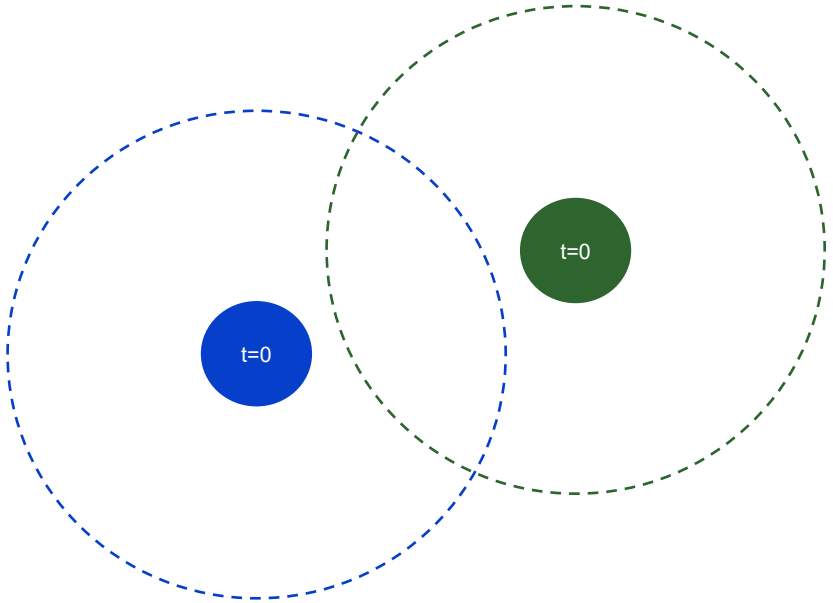


This is the liquid Ar MD trajectory at 85K.

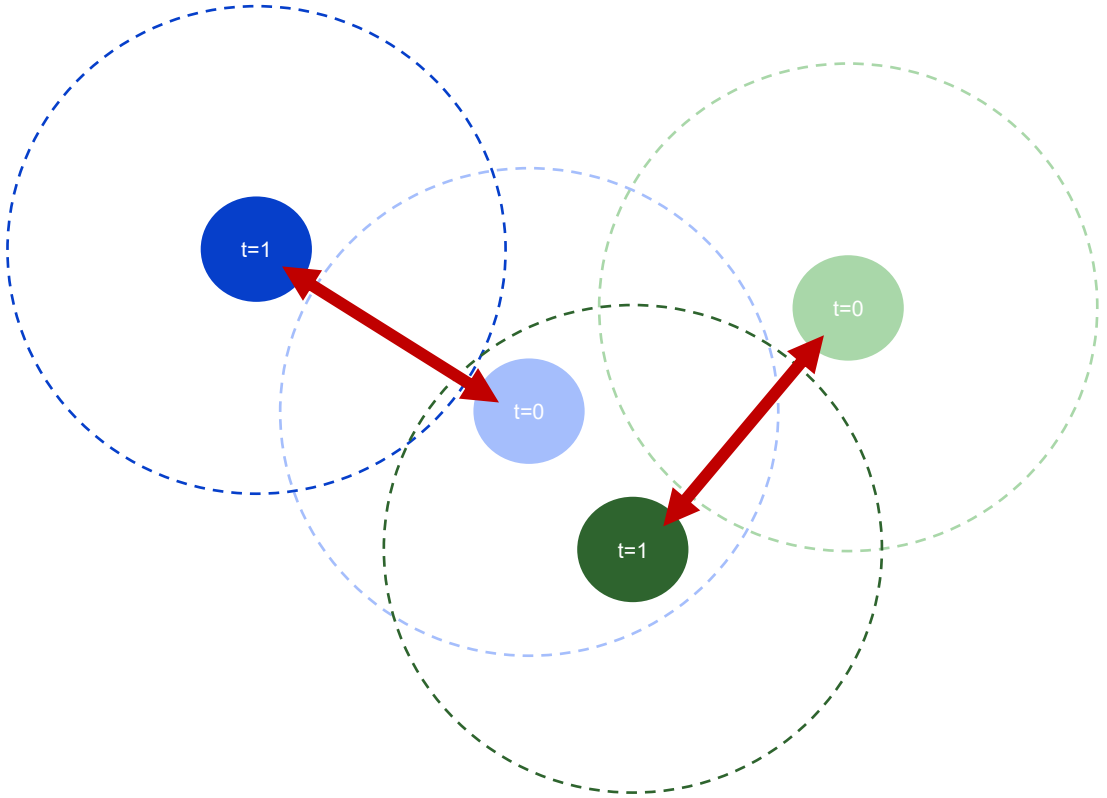


Van Hove Function (self-part)

$$G_s(\mathbf{r}, t = 0)$$



$$G_s(\mathbf{r}, t = 1)$$

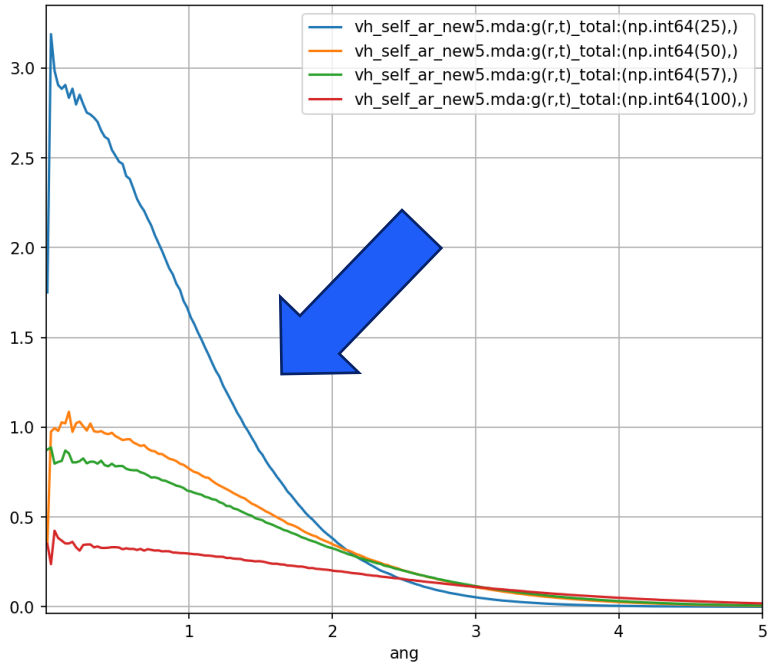


Van Hove Function (self-part)

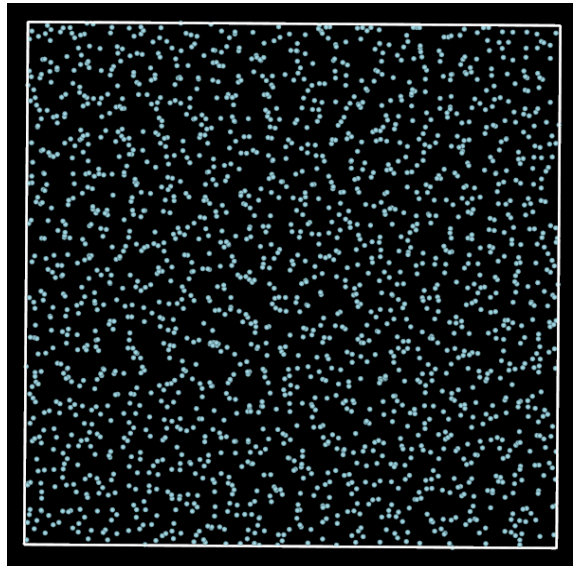
- The self-part of the van Hove function can be written so that it is only a function of the distance r.

$$G_s(r,t) = \frac{1}{4\pi r^2}\frac{1}{N}\sum_j \langle \delta[r - |\mathbf{r}_j(t_0+t) - \mathbf{r}_j(t_0)|]\rangle_{t_0}$$

The Ar atoms diffuse away from its initial position. The self-part of the van Hove function moves and spreads out as time progresses.



This is the liquid Ar MD trajectory at 85K.

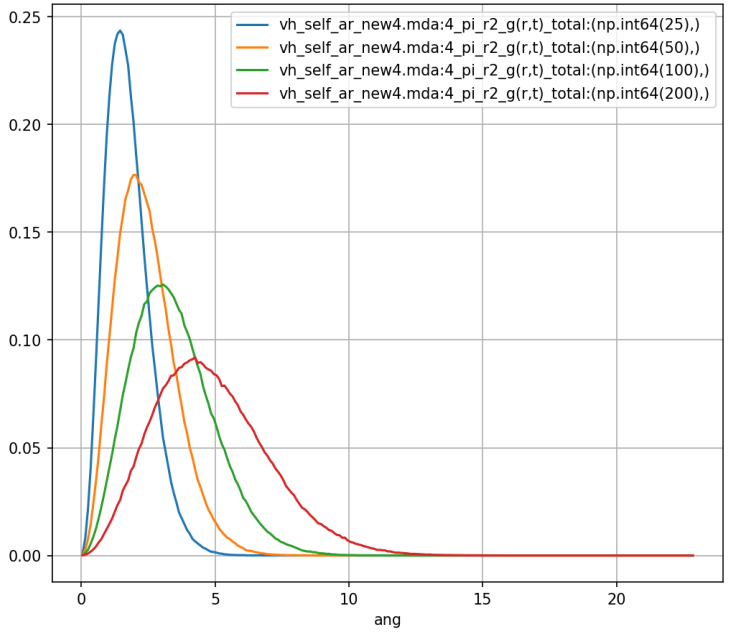


Van Hove Function (self-part)

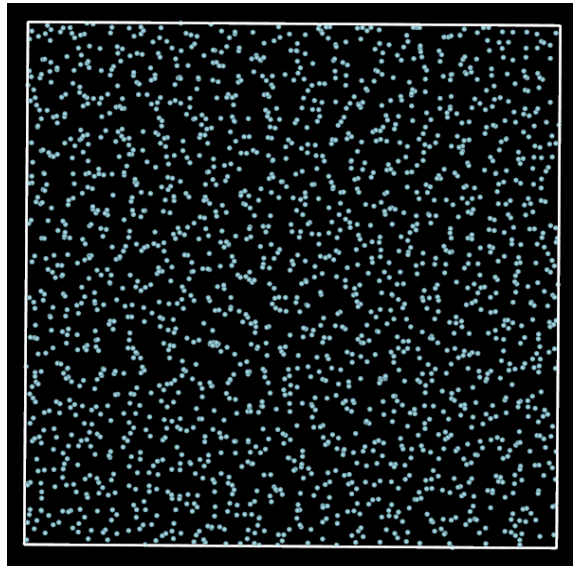
- The self-part of the van Hove function can be written so that it is only a function of the distance r.

$$4\pi r^2 G_s(r,t) = \frac{1}{N}\sum_j \langle \delta[r - |\mathbf{r}_j(t_0+t) - \mathbf{r}_j(t_0)|] \rangle_{t_0}$$

It is also useful to plot a normalized self-part of the van Hove function.



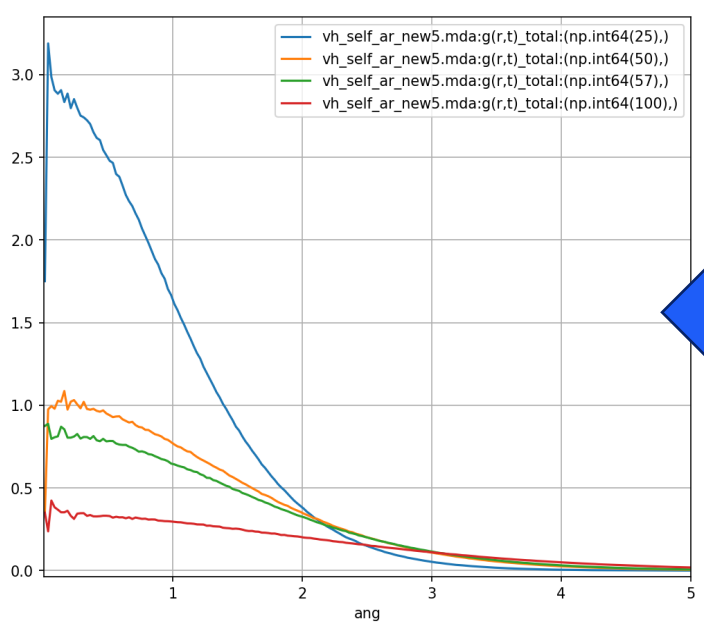
This is the liquid Ar MD trajectory at 85K.

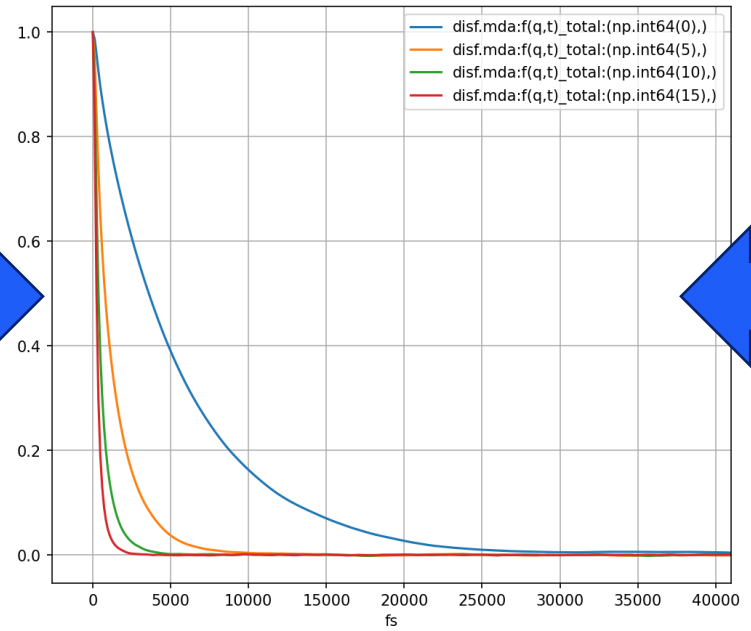


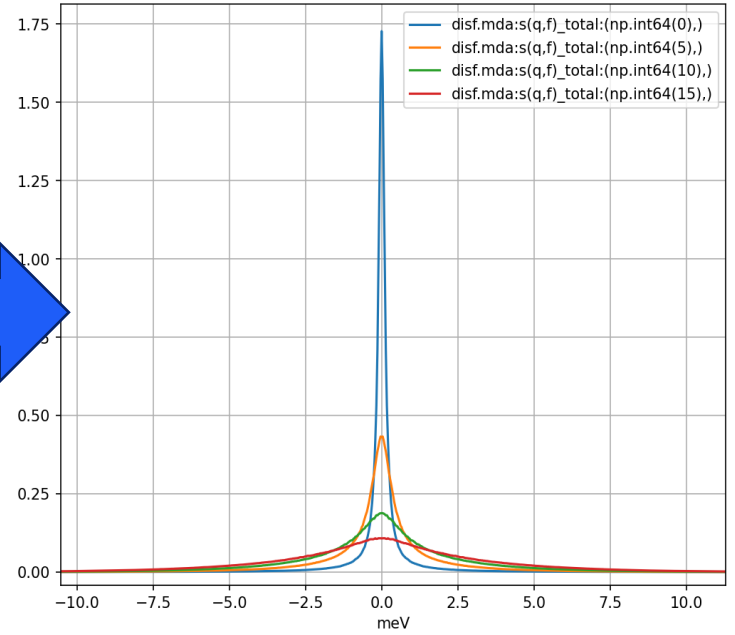
Van Hove Function (relation to neutron scattering)

- The van Hove function (and therefore the PDF) are closely related to the dynamics structure factor via Fourier transforms. Here we show the self-part and its Fourier transforms.

$G_s(\mathbf{r}, t)$ $F_s(\mathbf{Q}, t)$ $S_i(\mathbf{Q}, \omega)$







Water

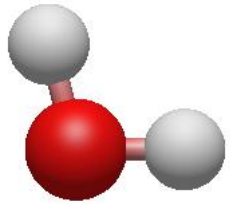
- In this workshop exercise, we will run simulation of water using OpenMM and then run van Hove calculations with MDANSE.
- Using a PDB file we will use PACKMOL to generate a box of water molecules.
- Actually, there is a function in OpenMM function `Modeller.addSolvent` which we could use to solvate an empty box with water.
- We will use PACKMOL as it is useful to learn how to use this program since you may want to create a box of other molecules or a mixture of different molecules in the future.

OpenMM



PACKMOL

- If the files you should see `water.pdb`. This is a PDB file containing a single water molecule.



		atom name	residue name			x position	y position	z position			element
ATOM	1	O	HOH	A	1	0.000	0.000	0.000	1.00	0.00	O
ATOM	2	H1	HOH	A	1	0.957	0.000	0.000	1.00	0.00	H
ATOM	3	H2	HOH	A	1	-0.240	0.927	0.000	1.00	0.00	H
END											

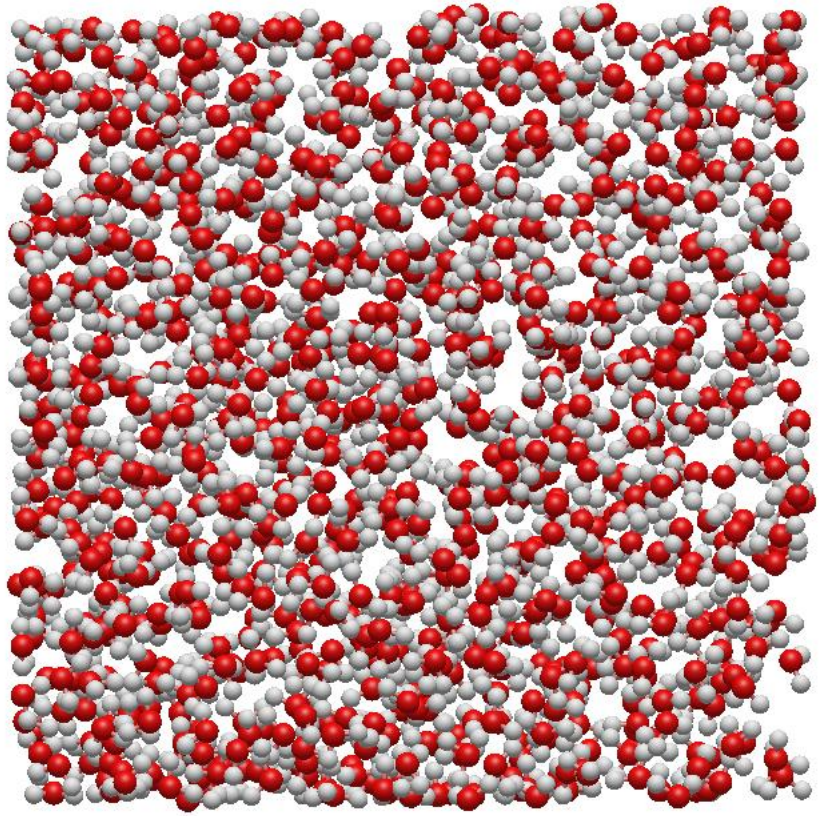
PACKMOL

- First let's install a few thing in a new conda environment.

```
conda create -n openmm python=3.13
conda activate openmm
pip install packmol
pip install openmm
pip install pandas
pip install matplotlib
```

- On windows you'll need `libgfortran-5.dll` you can install this with.

```
conda install -c conda-forge libgfortran5
```



PACKMOL

- In your files you should see another file called `water_packmol.inp` this is the input file we will use with packmol to generate a box of water molecules.

```
tolerance 2.0
filetype pdb
output water_box.pdb

structure water.pdb
  number 1000
  inside box 1.0 1.0 1.0 30.05 30.05 30.05
end structure
```

PACKMOL

- In your files you should see another file called `water_packmol.inp` this is the input file we will use with packmol to generate a box of water molecules.
- Tolerance the minimum distance between atoms in angstrom.



```
tolerance 2.0
filetype pdb
output water_box.pdb

structure water.pdb
  number 1000
  inside box 1.0 1.0 1.0 30.05 30.05 30.05
end structure
```

PACKMOL

- In your files you should see another file called `water_packmol.inp` this is the input file we will use with packmol to generate a box of water molecules.
- Tolerance the minimum distance between atoms in angstrom.
- Output file type and name.



```
tolerance 2.0
filetype pdb
output water_box.pdb

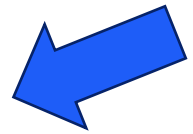
structure water.pdb
  number 1000
  inside box 1.0 1.0 1.0 30.05 30.05 30.05
end structure
```

PACKMOL

- In your files you should see another file called `water_packmol.inp` this is the input file we will use with packmol to generate a box of water molecules.
- Tolerance the minimum distance between atoms in angstrom.
- Output file type and name.
- Input file for one of the molecules. Here we have water and specified that we want 1000 of them.

```
tolerance 2.0
filetype pdb
output water_box.pdb

structure water.pdb
  number 1000
  inside box 1.0 1.0 1.0 30.05 30.05 30.05
end structure
```



PACKMOL

- `inside box` specifies the minimum and maximum xyz coordinates to place the molecules.
- Note that we have left a border where there will be no atoms. This is because PACKMOL does not have periodic boundaries while our MD simulation will. This will stop atoms from overlapping over each other when we apply periodic boundaries.
- The empty border will get filled in when we run our MD simulation.

```
tolerance 2.0
filetype pdb
output water_box.pdb

structure water.pdb
  number 1000
  inside box 1.0 1.0 1.0 30.05 30.05 30.05
end structure
```



PACKMOL

- You can run PACKMOL with this command. If you are using windows, make sure you are using cmd.

```
packmol < water_packmol.inp
```

- If it all runs correctly you should see this success message.

```
###############################################################################

                                  Success!
                 Final objective function value: .42726E+00
                 Maximum violation of target distance:   0.000000
                 Maximum violation of the constraints: .83285E-02

-------------------------------------------------------------------------------

                 Please cite this work if Packmol was useful:

             L. Martinez, R. Andrade, E. G. Birgin, J. M. Martinez,
           PACKMOL: A package for building initial configurations for
                      molecular dynamics simulations.
          Journal of Computational Chemistry, 30(13) pp. 2157-2164, 2009.
                    https://doi.org/10.1002/jcc.21224

###############################################################################

   Running time:   0.703125000      seconds.

-------------------------------------------------------------------------------
```

OpenMM

- Now we have a box of water molecules in the PDB file format.
- In the files there is a python script to run OpenMM with your PDB file. However, the script is not runnable and needs to be completed.
- If you see … this is where you will need to fill in to complete the script.

```python
import sys

import openmm as mm
import openmm.app as app
import openmm.unit as unit


PACKED_PDB = "water_box.pdb"

TEMPERATURE = 300 * unit.kelvin
PRESSURE = ...
TIMESTEP = ...

NPT_STEPS = int(round(100.0 / TIMESTEP.value_in_unit(unit.picosecond)))
NVT_STEPS = int(round(50.0 / TIMESTEP.value_in_unit(unit.picosecond)))
NVE_EQUIL_STEPS = int(round(25.0 / TIMESTEP.value_in_unit(unit.picosecond)))
NVE_PROD_STEPS = int(round(100.0 / TIMESTEP.value_in_unit(unit.picosecond)))
REPORT_INTERVAL = int(round(1.0 / TIMESTEP.value_in_unit(unit.picosecond)))
DCD_INTERVAl = int(round(20.0 / TIMESTEP.value_in_unit(unit.femtoseconds)))
```

OpenMM

- Now we have a box of water molecules in the PDB file format.
- In the files there is a python script to run OpenMM with your PDB file. However, the script is not runnable and needs to be completed.
- If you see … this is where you will need to fill in to complete the script.
- Here we have specified the temperature in units of kelvin.

```python
import sys

import openmm as mm
import openmm.app as app
import openmm.unit as unit


PACKED_PDB = "water_box.pdb"

TEMPERATURE = 300 * unit.kelvin
PRESSURE = ...
TIMESTEP = ...

NPT_STEPS = int(round(100.0 / TIMESTEP.value_in_unit(unit.picosecond)))
NVT_STEPS = int(round(50.0 / TIMESTEP.value_in_unit(unit.picosecond)))
NVE_EQUIL_STEPS = int(round(25.0 / TIMESTEP.value_in_unit(unit.picosecond)))
NVE_PROD_STEPS = int(round(100.0 / TIMESTEP.value_in_unit(unit.picosecond)))
REPORT_INTERVAL = int(round(1.0 / TIMESTEP.value_in_unit(unit.picosecond)))
DCD_INTERVAl = int(round(20.0 / TIMESTEP.value_in_unit(unit.femtoseconds)))
```

OpenMM

- Can you fill in the rest? We want to run our simulations at room temperature and pressure.
- For the time step should depend on the water model you want to use. If you want to run a flexible water model this should be small around 0.5 fs or smaller, for a rigid water model you can use a timestep of 2.0 fs or smaller.

```python
import sys

import openmm as mm
import openmm.app as app
import openmm.unit as unit


PACKED_PDB = "water_box.pdb"

TEMPERATURE = 300 * unit.kelvin
PRESSURE = ...
TIMESTEP = ...

NPT_STEPS = int(round(100.0 / TIMESTEP.value_in_unit(unit.picosecond)))
NVT_STEPS = int(round(50.0 / TIMESTEP.value_in_unit(unit.picosecond)))
NVE_EQUIL_STEPS = int(round(25.0 / TIMESTEP.value_in_unit(unit.picosecond)))
NVE_PROD_STEPS = int(round(100.0 / TIMESTEP.value_in_unit(unit.picosecond)))
REPORT_INTERVAL = int(round(1.0 / TIMESTEP.value_in_unit(unit.picosecond)))
DCD_INTERVAl = int(round(20.0 / TIMESTEP.value_in_unit(unit.femtoseconds)))
```

OpenMM

- In the next block we specify the number of time steps to run a simulation with 50, 25, and 10 ps for the NPT, NVT, and NVE equilibration stages and 100 ps for the NVE production.

```python
import sys

import openmm as mm
import openmm.app as app
import openmm.unit as unit


PACKED_PDB = "water_box.pdb"

TEMPERATURE = 300 * unit.kelvin
PRESSURE = ...
TIMESTEP = ...

NPT_STEPS = int(round(100.0 / TIMESTEP.value_in_unit(unit.picosecond)))
NVT_STEPS = int(round(50.0 / TIMESTEP.value_in_unit(unit.picosecond)))
NVE_EQUIL_STEPS = int(round(25.0 / TIMESTEP.value_in_unit(unit.picosecond)))
NVE_PROD_STEPS = int(round(100.0 / TIMESTEP.value_in_unit(unit.picosecond)))
REPORT_INTERVAL = int(round(1.0 / TIMESTEP.value_in_unit(unit.picosecond)))
DCD_INTERVAl = int(round(20.0 / TIMESTEP.value_in_unit(unit.femtoseconds)))
```

OpenMM

- In the next block we specify the number of time steps to run a simulation with 50, 25, and 10 ps for the NPT, NVT, and NVE equilibration stages and 100 ps for the NVE production.
- In this line we specify the report interval this will specify how often we will output the MD simulation information.

```
import sys

import openmm as mm
import openmm.app as app
import openmm.unit as unit


PACKED_PDB = "water_box.pdb"

TEMPERATURE = 300 * unit.kelvin
PRESSURE = ...
TIMESTEP = ...

NPT_STEPS = int(round(100.0 / TIMESTEP.value_in_unit(unit.picosecond)))
NVT_STEPS = int(round(50.0 / TIMESTEP.value_in_unit(unit.picosecond)))
NVE_EQUIL_STEPS = int(round(25.0 / TIMESTEP.value_in_unit(unit.picosecond)))
NVE_PROD_STEPS = int(round(100.0 / TIMESTEP.value_in_unit(unit.picosecond)))
REPORT_INTERVAL = int(round(1.0 / TIMESTEP.value_in_unit(unit.picosecond)))
DCD_INTERVAl = int(round(20.0 / TIMESTEP.value_in_unit(unit.femtoseconds)))
```

OpenMM

- In the next block we specify the number of time steps to run a simulation with 50, 25, and 10 ps for the NPT, NVT, and NVE equilibration stages and 100 ps for the NVE production.
- In this line we specify the report interval this will specify how often we will output the MD simulation information.
- Here we define the DCD interval which we use to tell the reporter how often to dump the trajectory results. We will do this every 20 fs.

```python
import sys

import openmm as mm
import openmm.app as app
import openmm.unit as unit

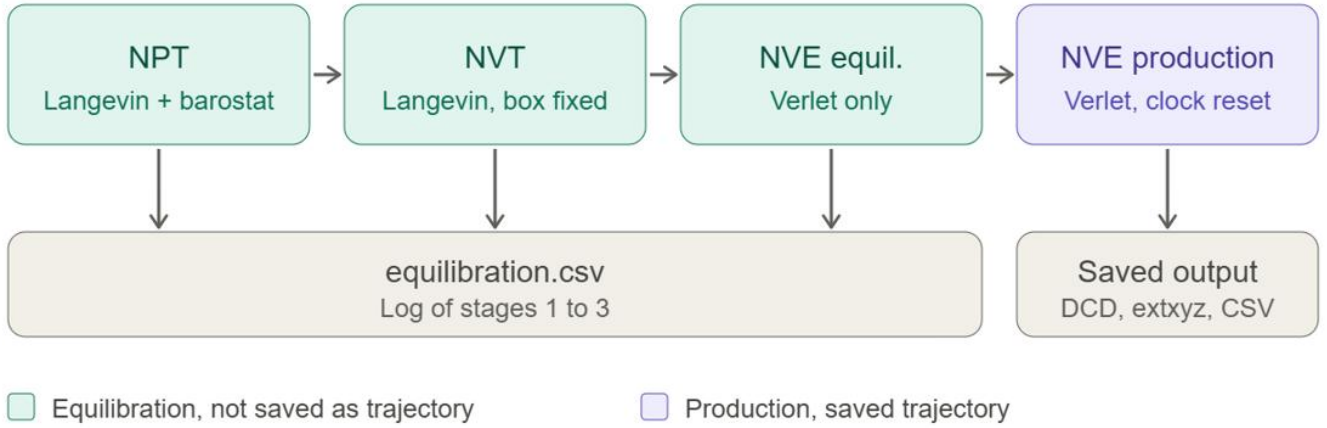

PACKED_PDB = "water_box.pdb"

TEMPERATURE = 300 * unit.kelvin
PRESSURE = ...
TIMESTEP = ...

NPT_STEPS = int(round(100.0 / TIMESTEP.value_in_unit(unit.picosecond)))
NVT_STEPS = int(round(50.0 / TIMESTEP.value_in_unit(unit.picosecond)))
NVE_EQUIL_STEPS = int(round(25.0 / TIMESTEP.value_in_unit(unit.picosecond)))
NVE_PROD_STEPS = int(round(100.0 / TIMESTEP.value_in_unit(unit.picosecond)))
REPORT_INTERVAL = int(round(1.0 / TIMESTEP.value_in_unit(unit.picosecond)))
DCD_INTERVAL = int(round(20.0 / TIMESTEP.value_in_unit(unit.femtoseconds)))
```

OpenMM

- So why are we running MD like this? In the next stages of the tutorial we will want to run distinct van-Hove function calculations which calculates distances between atoms at different times.
- A barostat can cause the distances an atom has travel over time can be distorted by unit cell fluctuations. Similarly, a thermostat can interfere with the atom velocities and therefore its trajectory. Remember that NPT/NVT is about generating the right sampling for there respective ensembles, the trajectories themselves will be distorted in some way.
- We therefore must use NVE for production. We run NPT to equilibrate the density, then fix the volume and equilibrate the temperature only with NVT, then a finally an NVE equilibration to prepare it ready for the NVE production.



OpenMM

- In the next part we load our PDB file and specify the size of our simulation cell.

```
# load the pdf file and set the box vectors
pdb = app.PDBFile(PACKED_PDB)
topology = pdb.topology
topology.setPeriodicBoxVectors(
    [mm.Vec3(3.105, 0, 0),
     mm.Vec3(0, 3.105, 0),
     mm.Vec3(0, 0, 3.105)] * unit.nanometer
)

# create the force field and add extra particles
forcefield = app.ForceField(...)
modeller = app.Modeller(pdb.topology, pdb.positions)
modeller.addExtraParticles(forcefield)
topology = modeller.topology
```

OpenMM

- In the next part we load our PDB file and specify the size of our simulation cell.
- Here, we specify the force field. Here you will choose the force field that you want to use. In OpenMM force fields are defined using an XML file. This contains all the force field parameters.
- You can give OpenMM your own custom force field parameters but it also comes with some standard force fields.
- Have a look at https://docs.openmm.org/7.6.0/userguide/application/02_running_sims.html#force-fields to see what sort of force fields you can use.

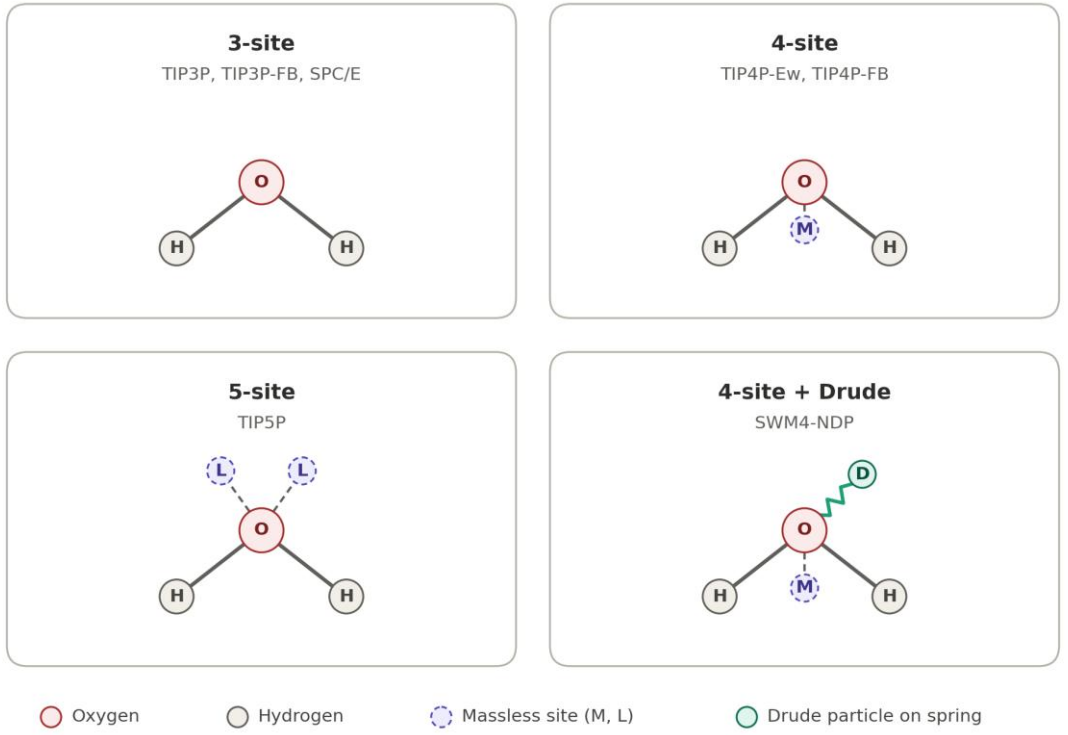
```
20  # load the pdf file and set the box vectors
21  pdb = app.PDBFile(PACKED_PDB)
22  topology = pdb.topology
23  topology.setPeriodicBoxVectors(
24      [mm.Vec3(3.105, 0, 0),
25       mm.Vec3(0, 3.105, 0),
26       mm.Vec3(0, 0, 3.105)] * unit.nanometer
27  )
28
29  # create the force field and add ext
30  forcefield = app.ForceField(...)
31  modeller = app.Modeller(pdb.topology, pdb.positions)
32  modeller.addExtraParticles(forcefield)
33  topology = modeller.topology
```

OpenMM

- Here are the water models that comes with OpenMM. They can be broadly grouped up into 3, 4, and 5-site models.
- TIP3P, TIP3P-FB, and SPC/E are 3-site water models.
- TIP4P, TIP4P-FB are 4-site water models.
- SWM4-NDP is a 4-site water model with a Drude particle.
- TIP5P is a 5-site water model.

File	Water Model
`tip3p.xml`	TIP3P water model[2]
`tip3pfb.xml`	TIP3P-FB water model[3]
`tip4pew.xml`	TIP4P-Ew water model[4]
`tip4pfb.xml`	TIP4P-FB water model[3]
`tip5p.xml`	TIP5P water model[9]
`spce.xml`	SPC/E water model[5]
`swm4ndp.xml`	SWM4-NDP water model[23]

https://docs.openmm.org/7.6.0/userguide/application/02_running_sims.html#water-models



OpenMM

- Here are the water models that comes with OpenMM. They can be broadly grouped up into 3, 4, and 5-site models.
- TIP3P, TIP3P-FB, and SPC/E are 3-site water models.
- TIP4P, TIP4P-FB are 4-site water models.
- SWM4-NDP is a 4-site water model with a Drude particle.
- TIP5P is a 5-site water model.
- Select the water model you want to use.

File	Water Model
`tip3p.xml`	TIP3P water model[2]
`tip3pfb.xml`	TIP3P-FB water model[3]
`tip4pew.xml`	TIP4P-Ew water model[4]
`tip4pfb.xml`	TIP4P-FB water model[3]
`tip5p.xml`	TIP5P water model[9]
`spce.xml`	SPC/E water model[5]
`swm4ndp.xml`	SWM4-NDP water model[23]

https://docs.openmm.org/7.6.0/userguide/application/02_running_sims.html#water-models

```
20  # load the pdf file and set the box vectors
21  pdb = app.PDBFile(PACKED_PDB)
22  topology = pdb.topology
23  topology.setPeriodicBoxVectors(
24      [mm.Vec3(3.105, 0, 0),
25       mm.Vec3(0, 3.105, 0),
26       mm.Vec3(0, 0, 3.105)] * unit.nanometer
27  )
28
29  # create the force field and add ext
30  forcefield = app.ForceField(...)
31  modeller = app.Modeller(pdb.topology, pdb.positions)
32  modeller.addExtraParticles(forcefield)
33  topology = modeller.topology
```

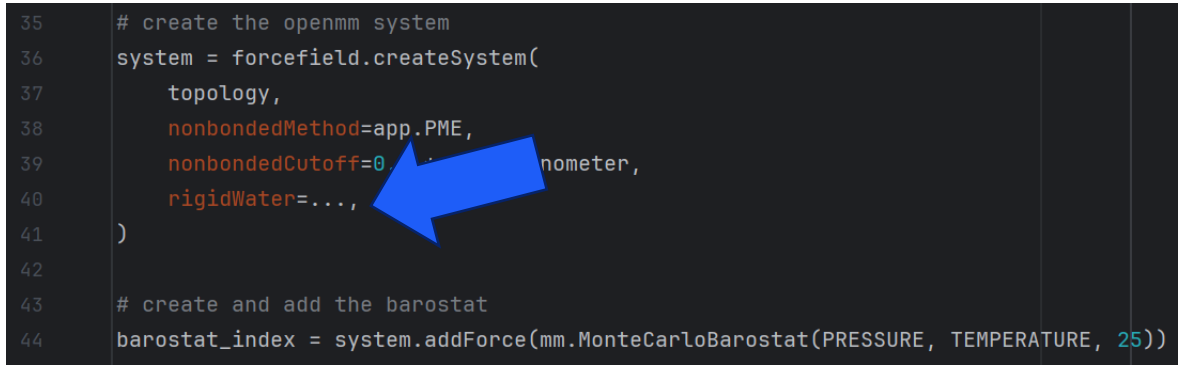
OpenMM

- In the next section, we create the OpenMM system. Here we also specify the method to sum the nonbonded interactions.
- In OpenMM the nonbonded interactions are the lennard jones plus coulomb interactions. Here we use particle mesh Ewald for the long-range electrostatics and a cutoff of 0.8 nm for the direct space interactions.

```
35  # create the openmm system
36  system = forcefield.createSystem(
37      topology,
38      nonbondedMethod=app.PME,
39      nonbondedCutoff=0.8 * unit.nanometer,
40      rigidWater=...,
41  )
42
43  # create and add the barostat
44  barostat_index = system.addForce(mm.MonteCarloBarostat(PRESSURE, TEMPERATURE, 25))
```

OpenMM

- In the next section, we create the OpenMM system. Here we also specify the method to sum the nonbonded interactions.
- In OpenMM the nonbonded interactions are the lennard jones plus coulomb interactions. Here we use particle mesh Ewald for the long-range electrostatics and a cutoff of 0.8 nm for the direct space interactions.
- Here specify whether you want to run a rigid or flexible water models using `True` or `False`.



OpenMM

- Here we add a barostat to our simulation, it's the like the Monte-Carlo barostat you might have coded up yesterday if you had the chance.
- In the MonteCarloBarostat we input the target pressure and temperature as well as the MonteCarlo pressure change frequency. Here we use the same values as the default, so that there is a Monte-Carlo volume change every 25 time steps.

```
35  # create the openmm system
36  system = forcefield.createSystem(
37      topology,
38      nonbondedMethod=app.PME,
39      nonbondedCutoff=0.8 * unit.nanometer,
40      rigidWater=...,
41  )
42
43  # create and add the barostat
44  barostat_index = system.addForce(mm.MonteCarloBarostat(PRESSURE, TEMPERATURE, 25))
```

OpenMM

- Next, we create two integrators one for our NPT and NVT runs and one for our NVE run.
- Let's first look at the integrator we could use for our NPT and NVT runs. Go to https://docs.openmm.org/latest/userguide/theory/04_integrators.html.
- There are a few we can choose from. Remember that we need an integrator which samples at constant temperature. We need something other than the Verlet integrator which generates trajectories at constant energy.

```
46  # create the integrators for our NPT, NVT, and NVT runs
47  nvt_integrator = ...
48  nve_integrator = mm.VerletIntegrator(TIMESTEP)
49  nve_integrator.setConstraintTolerance(1e-6)
50  integrator = mm.CompoundIntegrator()
51  integrator.addIntegrator(nvt_integrator)
52  integrator.addIntegrator(nve_integrator)
53  integrator.setCurrentIntegrator(0)  # set to the NVT integrator
```

OpenMM

- For our case we should choose between the LangevinIntegator and the NoseHooverIntegrator.
- Take a look at https://docs.openmm.org/latest/api-python/generated/openmm.openmm.LangevinIntegrator.html or https://docs.openmm.org/latest/api-python/generated/openmm.openmm.NoseHooverIntegrator.html to see how to add them to our script.

```
46    # create the integrators for our NPT, NVT, and NVT runs
47    nvt_integrator = ...
48    nve_integrator = mm.VerletIntegrator(TIMESTEP)
49    nve_integrator.setConstraintTolerance(1e-6)
50    integrator = mm.CompoundIntegrator()
51    integrator.addIntegrator(nvt_integrator)
52    integrator.addIntegrator(nve_integrator)
53    integrator.setCurrentIntegrator(0)  # set to the NVT integrator
```

OpenMM

- Next, we create the Verlet integrator and the compound integrator and add both the NPT/NVT and Verlet integrators to it.
- The compound integrator allows use to switch between different integrators during our MD simulation.
- We start by setting the current integrator to the NPT/NVT integrator since we want to start the equilibration with an NPT run.

```
46  # create the integrators for our NPT, NVT, and NVT runs
47  nvt_integrator = ...
48  nve_integrator = mm.VerletIntegrator(TIMESTEP)
49  nve_integrator.setConstraintTolerance(1e-6)
50  integrator = mm.CompoundIntegrator()
51  integrator.addIntegrator(nvt_integrator)
52  integrator.addIntegrator(nve_integrator)
53  integrator.setCurrentIntegrator(0)  # set to the NVT integrator
```



OpenMM

- In this part, of the script we create the OpenMM simulation and set starting positions and velocities.

```
55  # create the openmm simulation and set the positions and velocities
56  simulation = app.Simulation(topology, system, integrator)
57  simulation.context.setPositions(modeller.positions)
58  simulation.context.setVelocitiesToTemperature(TEMPERATURE)
59
60  # create some reporters which outputs some details of the simulation
61  simulation.reporters.append(app.StateDataReporter(
62      "equilibration.csv", REPORT_INTERVAL, step=True, time=True,
63      potentialEnergy=True, totalEnergy=True, temperature=True, volume=True,
64      density=True, separator=","))
65  simulation.reporters.append(app.StateDataReporter(
66      sys.stdout, REPORT_INTERVAL, step=True, temperature=True,
67      density=True, speed=True))
```

OpenMM

- In this part, of the script we create the OpenMM simulation and set starting positions and velocities.
- In the next part we create the two reporters and add them to the simulations object. We create one which adds simulation information to a csv file and another which prints the information out to the terminal.

```
55  # create the openmm simulation and set the positions and velocities
56  simulation = app.Simulation(topology, system, integrator)
57  simulation.context.setPositions(modeller.positions)
58  simulation.context.setVelocitiesToTemperature(TEMPERATURE)
59
60  # create some reporters which outputs some details of the simulation
61  simulation.reporters.append(app.StateDataReporter(
62      "equilibration.csv", REPORT_INTERVAL, step=True, time=True,
63      potentialEnergy=True, totalEnergy=True, temperature=True, volume=True,
64      density=True, separator=","))
65  simulation.reporters.append(app.StateDataReporter(
66      sys.stdout, REPORT_INTERVAL, step=True, temperature=True,
67      density=True, speed=True))
```

OpenMM

- Finally, we can now start the MD run. Here we start with an energy minimisation, this is important if we have atom which are too close this stops the MD from blowing up straight away.



```
69  # minimisation
70  print(f"Running OpenMM under the platform: {simulati      xt.getPlatform().getName()}")
71  simulation.minimizeEnergy(maxIterations=5000)
72
73  # NPT equilibration
74  print(f"Running NPT equilibration for: {NPT_STEPS} steps")
75  simulation.step(NPT_STEPS)
76
77  # NVT equilibration
78  print(f"Running NVT equilibration for: {NVT_STEPS} steps")
79  system.getForce(barostat_index).setFrequency(0)  # 0 disables the barostat
80  simulation.context.reinitialize(preserveState=True)  # we modified the force, we have to reinitialize
81  simulation.step(NVT_STEPS)
82
83  # NVE equilibration
84  print(f"Running NVE equilibration for: {NVE_PROD_STEPS} steps")
85  integrator.setCurrentIntegrator(1)  # switch to the velocity verlet
86  simulation.step(NVE_EQUIL_STEPS)
```

OpenMM

- Finally, we can now start the MD run. Here we start with an energy minimisation, this is important if we have atom which are too close this stops the MD from blowing up straight away.
- Next, we run NPT to equilibrate the volume and temperatures.

```
69  # minimisation
70  print(f"Running OpenMM under the platform: {simulation.context.getPlatform().getName()}")
71  simulation.minimizeEnergy(maxIterations=5000)
72
73  # NPT equilibration
74  print(f"Running NPT equilibration for: {NPT_STEPS} steps")
75  simulation.step(NPT_STEPS)
76
77  # NVT equilibration
78  print(f"Running NVT equilibration for: {NVT_STEPS} steps")
79  system.getForce(barostat_index).setFrequency(0)  # 0 disables the barostat
80  simulation.context.reinitialize(preserveState=True)  # we modified the force, we have to reinitialize
81  simulation.step(NVT_STEPS)
82
83  # NVE equilibration
84  print(f"Running NVE equilibration for: {NVE_PROD_STEPS} steps")
85  integrator.setCurrentIntegrator(1)  # switch to the velocity verlet
86  simulation.step(NVE_EQUIL_STEPS)
```

OpenMM

- Finally, we can now start the MD run. Here we start with an energy minimisation, this is important if we have atom which are too close this stops the MD from blowing up straight away.
- Next, we run NPT to equilibrate the volume and temperatures.
- Then we run NVT by turning the barostat off to further equilibrate the temperature at a constant volume.

```
69  # minimisation
70  print(f"Running OpenMM under the platform: {simulation.context.getPlatform().getName()}")
71  simulation.minimizeEnergy(maxIterations=5000)
72
73  # NPT equilibration
74  print(f"Running NPT equilibration for: {NPT_STEPS} steps")
75  simulation.step(NPT_STEPS)
76
77  # NVT equilibration
78  print(f"Running NVT equilibration for: {NVT_STEPS} steps")
79  system.getForce(barostat_index).setFrequency(0)  # 0 disables the barostat
80  simulation.context.reinitialize(preserveState=True)  # we modified the force, we have to reinitialize
81  simulation.step(NVT_STEPS)
82
83  # NVE equilibration
84  print(f"Running NVE equilibration for: {NVE_PROD_STEPS} steps")
85  integrator.setCurrentIntegrator(1)  # switch to the velocity verlet
86  simulation.step(NVE_EQUIL_STEPS)
```

OpenMM

- Finally, we can now start the MD run. Here we start with an energy minimisation, this is important if we have atom which are too close this stops the MD from blowing up straight away.
- Next, we run NPT to equilibrate the volume and temperatures.
- Then we run NVT by turning the barostat off to further equilibrate the temperature at a constant volume.
- Then we switch to the NVE integrator to equilibrate at constant volume and energy.

```
69  # minimisation
70  print(f"Running OpenMM under the platform: {simulation.context.getPlatform().getName()}")
71  simulation.minimizeEnergy(maxIterations=5000)
72
73  # NPT equilibration
74  print(f"Running NPT equilibration for: {NPT_STEPS} steps")
75  simulation.step(NPT_STEPS)
76
77  # NVT equilibration
78  print(f"Running NVT equilibration for: {NVT_STEPS} steps")
79  system.getForce(barostat_index).setFrequency(0)  # 0 disables the barostat
80  simulation.context.reinitialize(preserveState=True)  # we modified the force, we have to reinitialize
81  simulation.step(NVT_STEPS)
82
83  # NVE equilibration
84  print(f"Running NVE equilibration for: {NVE_PROD_STEPS} steps")
85  integrator.setCurrentIntegrator(1)  # switch to the velocity verlet
86  simulation.step(NVE_EQUIL_STEPS)
```

OpenMM

- Now we run our production run. First, we clear the reporters and add new ones including a DCD reporter which will output the trajectory into a DCD file.

```
98  # NVE production
99  print(f"Running NVE production for: {NVE_PROD_STEPS} steps")
100 # clear and create new reporters for the production stage
101 simulation.reporters.clear()
102 simulation.reporters.append(app.DCDReporter("production.dcd", REPORT_INTERVAL / 100))
103 simulation.reporters.append(app.StateDataReporter(
104     "production.csv", REPORT_INTERVAL, step=True, time=True,
105     potentialEnergy=True, kineticEnergy=True, totalEnergy=True,
106     temperature=True, volume=True, density=True, separator=","))
107 simulation.reporters.append(app.StateDataReporter(
108     sys.stdout, REPORT_INTERVAL, step=True, temperature=True,
109     totalEnergy=True, speed=True, totalSteps=NVE_PROD_STEPS,
110     progress=True))
111 # reset the time back to zero and run the production stage
112 simulation.context.setStepCount(0)
113 simulation.context.setTime(0 * unit.picosecond)
114 simulation.step(NVE_PROD_STEPS)
115
116 # Save the final structure. This is the topology file needed to the trajectory
117 # conversion with the DCD converter. We need to use this one and not the
118 # input PDB file since we might have added some virtual particles.
119 state = simulation.context.getState(getPositions=True, enforcePeriodicBox=True)
120 with open("water_box_final.pdb", "w") as f:
121     app.PDBFile.writeFile(simulation.topology, state.getPositions(), f)
```

OpenMM

- Now we run our production run. First, we clear the reporters and add new ones including a DCD reporter which will output the trajectory into a DCD file.
- Here we reset the step and time counter and run our simulation.

```
98  # NVE production
99  print(f"Running NVE production for: {NVE_PROD_STEPS} steps")
100 # clear and create new reporters for the production stage
101 simulation.reporters.clear()
102 simulation.reporters.append(app.DCDReporter("production.dcd", REPORT_INTERVAL / 100))
103 simulation.reporters.append(app.StateDataReporter(
104     "production.csv", REPORT_INTERVAL, step=True, time=True,
105     potentialEnergy=True, kineticEnergy=True, totalEnergy=True,
106     temperature=True, volume=True, density=True, separator=","))
107 simulation.reporters.append(app.StateDataReporter(
108     sys.stdout, REPORT_INTERVAL, step=True, temperature=True,
109     totalEnergy=True, speed=True, totalSteps=NVE_PROD_STEPS,
110     progress=True))
111 # reset the time back to zero and run the production stage
112 simulation.context.setStepCount(0)
113 simulation.context.setTime(0 * unit.picosecond)
114 simulation.step(NVE_PROD_STEPS)
115
116 # Save the final structure. This is the topology file needed to the trajectory
117 # conversion with the DCD converter. We need to use this one and not the
118 # input PDB file since we might have added some virtual particles.
119 state = simulation.context.getState(getPositions=True, enforcePeriodicBox=True)
120 with open("water_box_final.pdb", "w") as f:
121     app.PDBFile.writeFile(simulation.topology, state.getPositions(), f)
```

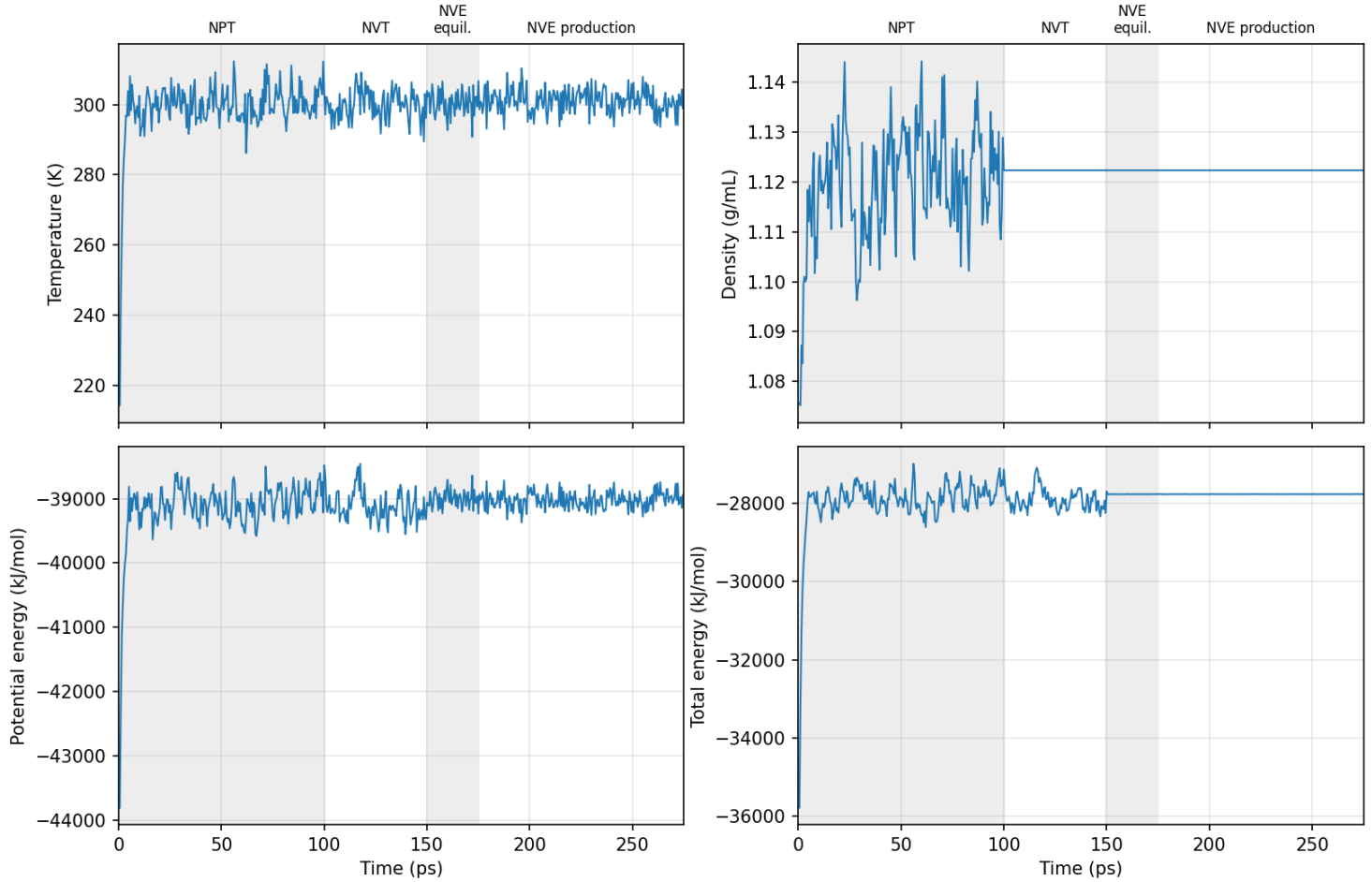
OpenMM

- Now we run our production run. First, we clear the reporters and add new ones including a DCD reporter which will output the trajectory into a DCD file.
- Here we reset the step and time counter and run our simulation.
- Here we write a PDB file for the final step. We need this to convert the file with MDANSE.

```
98  # NVE production
99  print(f"Running NVE production for: {NVE_PROD_STEPS} steps")
100 # clear and create new reporters for the production stage
101 simulation.reporters.clear()
102 simulation.reporters.append(app.DCDReporter("production.dcd", REPORT_INTERVAL / 100))
103 simulation.reporters.append(app.StateDataReporter(
104     "production.csv", REPORT_INTERVAL, step=True, time=True,
105     potentialEnergy=True, kineticEnergy=True, totalEnergy=True,
106     temperature=True, volume=True, density=True, separator=","))
107 simulation.reporters.append(app.StateDataReporter(
108     sys.stdout, REPORT_INTERVAL, step=True, temperature=True,
109     totalEnergy=True, speed=True, totalSteps=NVE_PROD_STEPS,
110     progress=True))
111 # reset the time back to zero and run the production stage
112 simulation.context.setStepCount(0)
113 simulation.context.setTime(0 * unit.picosecond)
114 simulation.step(NVE_PROD_STEPS)
115
116 # Save the final structure. This is the topology file needed to the trajectory
117 # conversion with the DCD converter. We need to use this one and not the
118 # input PDB file since we might have added some virtual particles.
119 state = simulation.context.getState(getPositions=True, enforcePeriodicBox=True)
120 with open("water_box_final.pdb", "w") as f:
121     app.PDBFile.writeFile(simulation.topology, state.getPositions(), f)
```

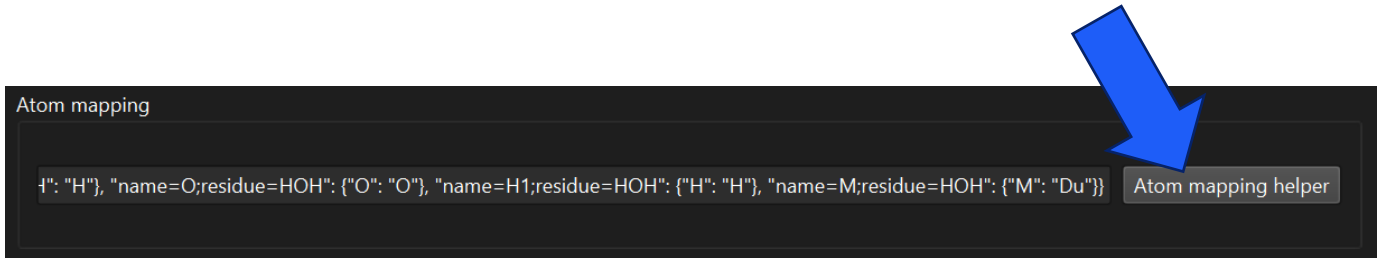
OpenMM

- Try running the openmm script once you have finished. It should generate a few files, equilibration.csv, production.csv, and production.csv.
- In the files there is a script called `csv_plot.py` you can use this to plot the MD run information.
- If everything ran well, you should see a plot similar to the on one the right.
- Generally, we don't want to see any sort of drift in the results.



MDANSE

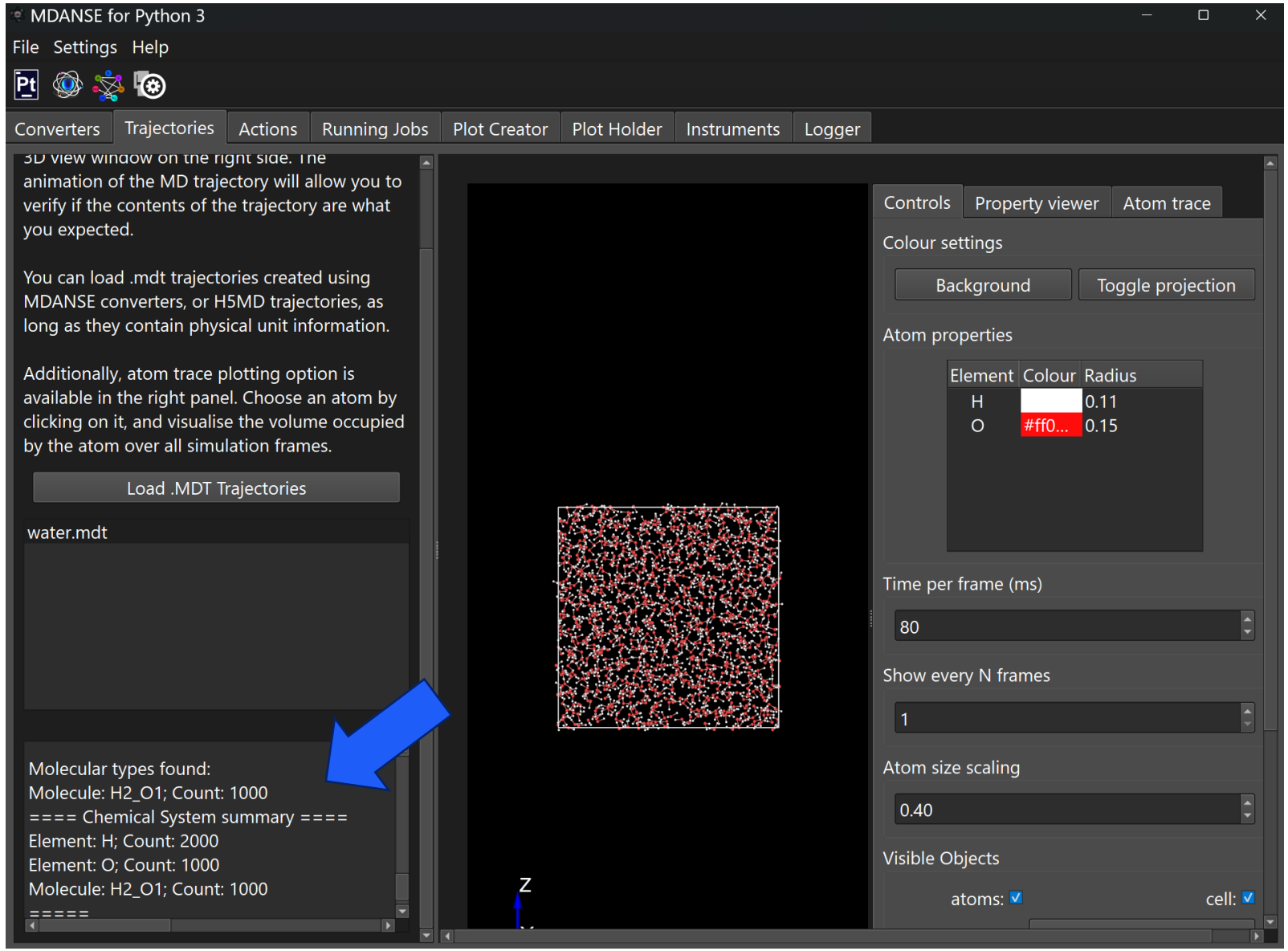
- Convert the DCD and PDB files to the MDANSE MDT format using the DCD converter. Make sure you set the correct time step.
- If used a water model with dummy atoms or etc, check that the correct atom types were assigned.





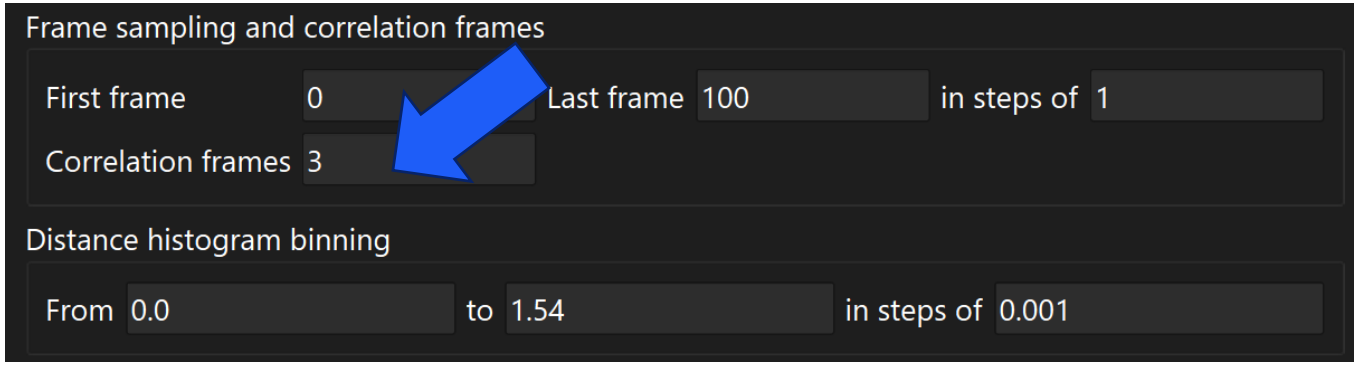
MDANSE

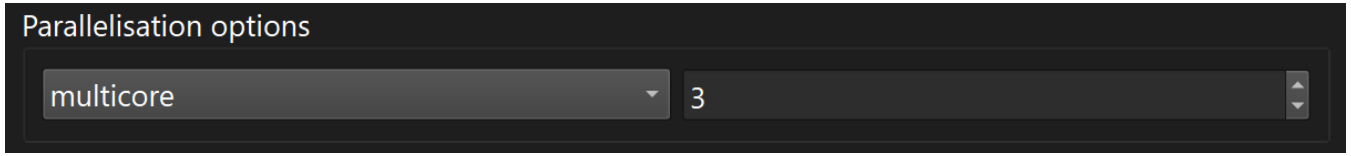
- Once complete view your trajectory in the trajectory tab and scroll down in the information box.
- Notice that MDANSE has grouped your atom into molecules. This is because the PDB file included some information which allows the DCD converter to group the atoms.



MDANSE

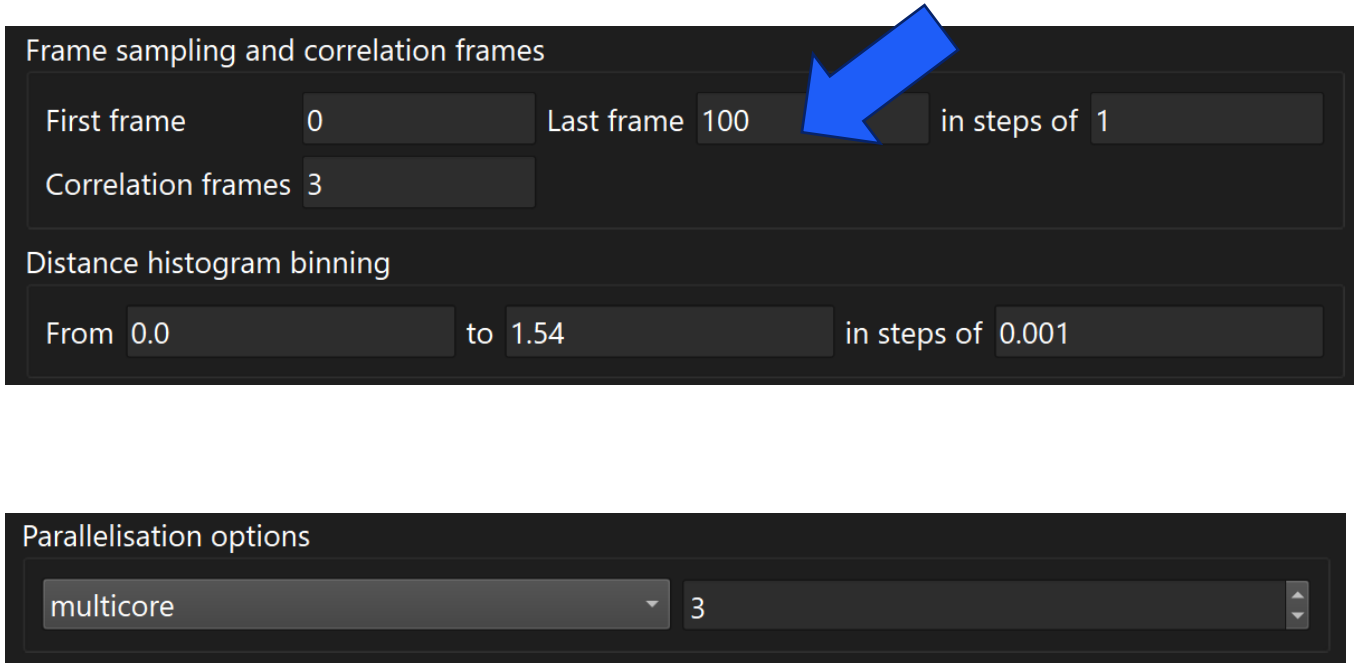
- Now go to the actions tab and select VanHoveFunctionDistinct under analysis and dynamics.
- The VanHoveFunctionDistinct is quite computationally expensive so let's pick specific settings to get the time differences we want.
- We set correlation frames to 3 so that we have results for t=0, 20, and 40 fs.





MDANSE

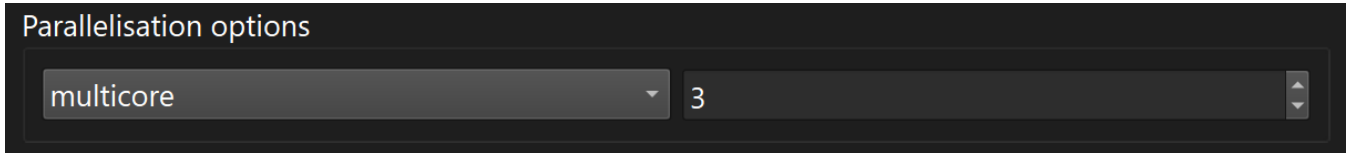
- Now go to the actions tab and select VanHoveFunctionDistinct under analysis and dynamics.
- The VanHoveFunctionDistinct is quite computationally expensive so let's pick specific settings to get the time differences we want.
- We set correlation frames to 3 so that we have results for t=0, 20, and 40 fs.
- We set the in frames of to 100 to reduce the calculation time a bit.



MDANSE

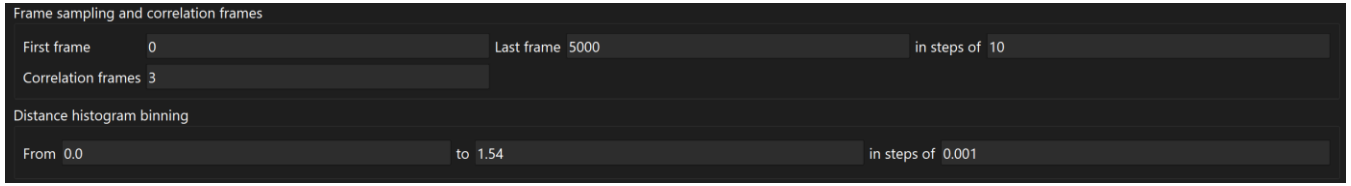
- Here we decrease the in step of setting to increase the resolution of the plots.





MDANSE

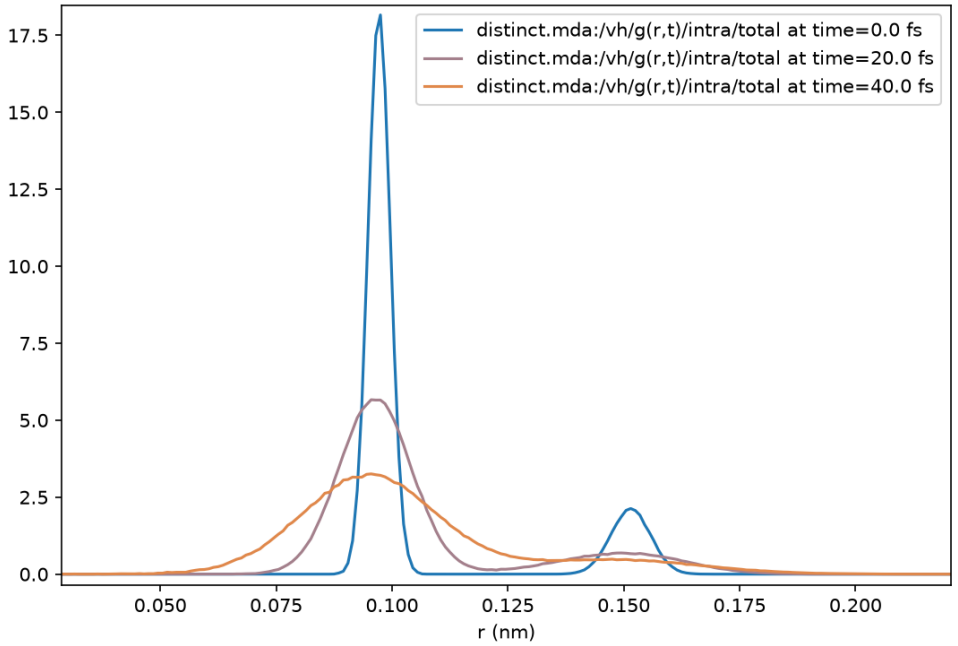
- Here we decrease the in step of setting to increase the resolution of the plots.
- To speed up the calculations you can increase the number of processor used. For van Hove distinct you should set this to be smaller than the number of correlation frames.

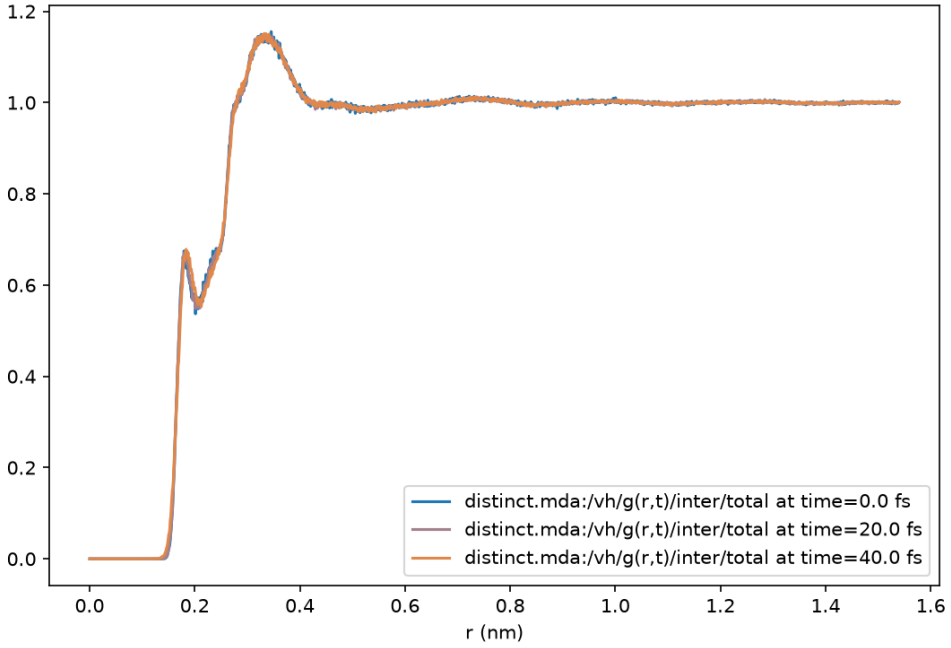




MDANSE

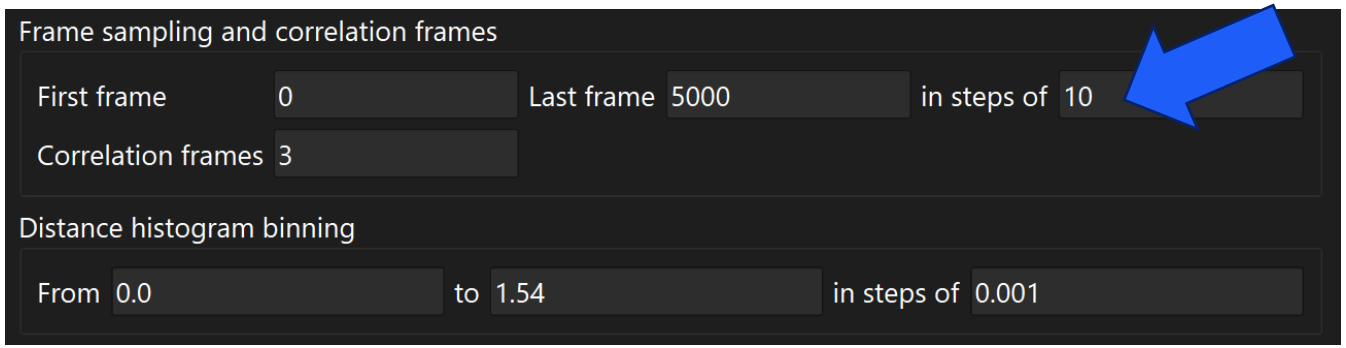
- Go to the plot creator/holder and plot your results. Since there were molecules in the trajectory the results should also be split into both intra and inter molecular parts.
- Have a look at the different results can you explain their behaviours?
- For example, can you explain why at this time scale the intermolecular results are mostly unchanged while the peaks in the intramolecular results have broaden out?
- For the intramolecular results can you see which peak belong to which pair of atoms?

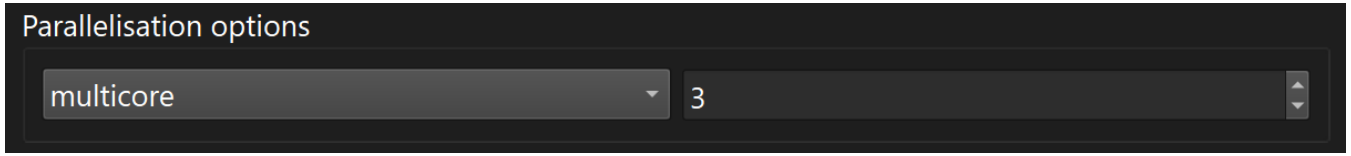




MDANSE

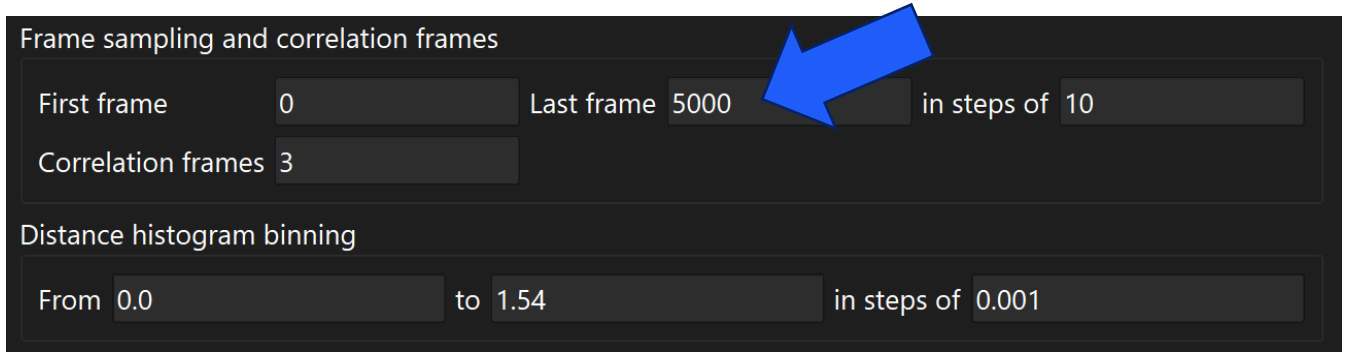
- Let's run the same calculation again but with a larger value for in steps of. This will mean that we will run calculate the van Hove function for times t = 0, 200, and 400 fs.

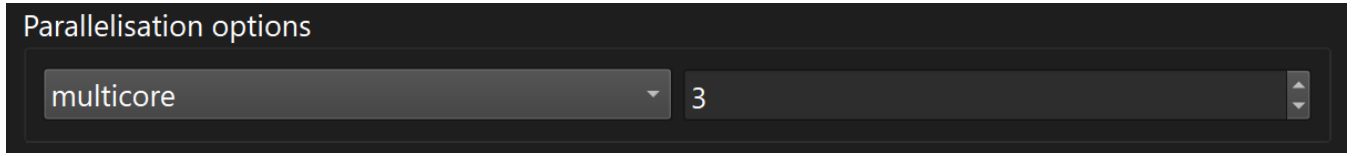




MDANSE

- Let's run the same calculation again but with a larger value for in steps of. This will mean that we will run calculate the van Hove function for times t = 0, 200, and 400 fs.
- Increase last frames to 5000, if you have a fast computer, this will decrease some of the noise in the results a bit. This should take around 10 mins on a fast laptop.





MDANSE

- Plot the results like last time. Results are a bit noisy, we could use a bigger system and/or longer trajectory to improve this, but this would take more time.
- You can see at this time scale, the intermolecular contribution to the distinct part of the van Hove function begin to change. What is happening for this to occur?

